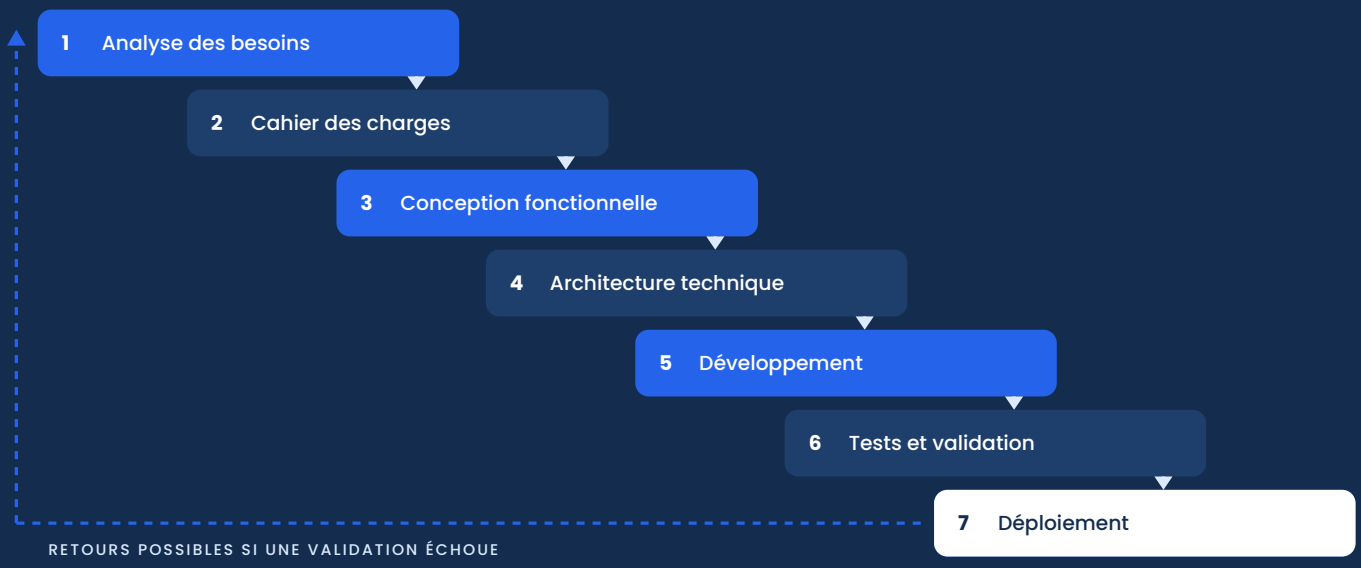
M.D.S

CREATED BY MAHDI CHERIF

M.D.S — PFE SERIES — ÉDITION 02

RÉALISER SON PROJET PFE

Guide pratique de conception et de développement logiciel avec la méthode en cascade (Waterfall Model).



MAHDI CHERIF

M.D.S — Created by Mahdi Cherif

ÉDITION 02

M.D.S
PFE SERIES — ÉDITION 02

RÉALISER SON PROJET PFE

Guide pratique de conception et de développement logiciel avec la méthode en cascade (Waterfall Model).

Collection : M.D.S — PFE SERIES — Édition 02
Auteur et créateur : Mahdi Cherif
Signature : M.D.S — Created by Mahdi Cherif
Complément de : « Réussir son PFE — De la recherche de stage à la soutenance » (Édition 01)

© Tous droits réservés — M.D.S — Mahdi Cherif

Aucune partie de ce guide ne peut être reproduite, diffusée ou vendue sans l'accord écrit de son auteur. Les modèles sont fournis pour un usage personnel : adaptez-les à votre projet.

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Nature de ce guide

Ce guide est un outil d'accompagnement. Il ne remplace ni le règlement de votre établissement, ni les consignes de vos encadrants. Le **projet fil rouge « SupportIT » est un cas pédagogique fictif** : l'entreprise, les personnes, les chiffres et les exemples de code ne représentent aucun déploiement réel.

« M.D.S » est une identité typographique créée à partir des initiales de la marque ; ce n'est pas un logo officiel déposé. Les noms de produits et de technologies appartiennent à leurs propriétaires. Les technologies évoluent : consultez toujours la documentation officielle et vérifiez les versions avant de les utiliser.

Avant-propos

Le premier guide de la collection expliquait comment réussir son PFE de bout en bout. Celui-ci répond à une question plus technique : **comment réaliser concrètement le projet, phase après phase, avec la méthode en cascade.**

La méthode en cascade ne se résume pas à une suite de tâches de programmation. Elle repose sur des **phases** qui produisent chacune des **livrables documentés**, sur des **validations** avant le passage à l'étape suivante et sur la **traçabilité** des exigences, du besoin initial jusqu'au test qui le vérifie. Ce guide vous montre comment faire, quels documents produire et comment valider chaque étape.

Le projet fil rouge : SupportIT

Tout le guide s'appuie sur un même exemple : **SupportIT**, une application web de gestion des interventions informatiques d'une entreprise fictive, *Atlas Services*. Trois acteurs (administrateur, technicien, employé), neuf fonctionnalités et une stack cohérente : **React.js, Node.js, Express.js, MySQL et JWT**. Vous suivrez le projet de l'analyse du besoin jusqu'au déploiement et à la documentation.

Chap.	Phase de la cascade	Livrable principal
1	Comprendre la méthode	—
2	Phase 1 — Étude préliminaire et analyse des besoins	Document d'analyse des besoins
3	Phase 2 — Cahier des charges et spécifications	Cahier des charges validé
4	Phase 3 — Conception fonctionnelle et UML	Dossier de conception fonctionnelle
5	Phase 4 — Conception technique et architecture	Dossier de conception technique
6	Phase 5 — Environnement et développement	Code source versionné
7	Phase 6 — Tests et validation	Rapport de tests et validation
8	Phase 7 — Déploiement et livraison	Application déployée et documentation
9	Documentation et rapport final	Dossier documentaire complet
10	Gestion de projet et conseils	—

Les encadrés du guide

Conseil professionnel — une pratique de métier à adopter.

Attention — un piège ou un risque à éviter.

Exemple pratique — une illustration tirée de SupportIT.

Définition — un terme expliqué avant d'être utilisé.

À vérifier — une exigence propre à votre établissement.

À retenir — l'essentiel du chapitre.

Sommaire

Chapitre 01 — Comprendre la méthode en cascade 7

1.1 Définition et principe de fonctionnement 8
1.2 Historique et origine de la méthode 9
1.3 Pourquoi l'utiliser dans un PFE ? 9
1.4 Caractéristiques, avantages, inconvénients et limites 10
1.5 Waterfall et Agile / Scrum : comparaison 10
1.6 Dans quels contextes choisir la cascade ? 11

Chapitre 02 — Phase 1 : étude préliminaire et analyse des besoins 13

2.1 Identifier le problème et comprendre le contexte 14
2.2 Identifier les utilisateurs et les parties prenantes 15
2.3 Recueillir les besoins 15
2.4 Organiser les réunions avec le client ou l'encadrant 16
2.5 Définir objectifs et périmètre 16
2.6 Contraintes et faisabilité 17
2.7 Besoins fonctionnels et non fonctionnels : SupportIT 18
2.8 Personas et acteurs 19
2.9 Livrable : le document d'analyse des besoins 19

Chapitre 03 — Phase 2 : cahier des charges et spécifications 21

3.1 Besoin, exigence, fonctionnalité, contrainte, critère d'acceptation 22
3.2 Structure d'un cahier des charges professionnel 22
3.3 Rédiger de bonnes exigences 23
3.4 Prioriser et tracer les exigences 23
3.5 Valider le cahier des charges 24

Chapitre 04 — Phase 3 : conception fonctionnelle et UML 26

4.1 UML : pourquoi et comment l'utiliser 27
4.2 Diagramme de cas d'utilisation 28
4.3 Spécifications fonctionnelles détaillées 30
4.4 Diagramme de classes 30
4.5 Diagramme de séquence 33
4.6 Diagramme d'activité 35
4.7 Diagramme d'états-transitions 38
4.8 Cohérence entre les diagrammes 39

Chapitre 05 — Phase 4 : conception technique et architecture 40

5.1 De la conception fonctionnelle à la conception technique 41
5.2 Les styles d'architecture 41
5.3 Choisir sa stack technologique 42
5.4 Architecture retenue pour SupportIT 43
5.5 Modélisation de la base de données 44

5.6 Conception de l'API REST . . . 46
5.7 Sécurité, authentification et rôles . . . 47
5.8 Organisation des dossiers du projet . . . 48

Chapitre 06 — Phase 5 : préparation de l'environnement et développement . . . 50

6.1 Installer et préparer les outils . . . 51
6.2 Configurer la base de données . . . 51
6.3 Initialiser le projet . . . 52
6.4 Organisation du code, conventions et variables d'environnement . . . 52
6.5 Développement back-end : API, base, erreurs . . . 53
6.6 Authentification JWT, validation et rôles . . . 55
6.7 Développement front-end avec React . . . 56
6.8 Bonnes pratiques Git . . . 57
6.9 Un dépôt GitHub professionnel . . . 58

Chapitre 07 — Phase 6 : tests et validation . . . 60

7.1 Pourquoi prévoir les tests dès la conception . . . 61
7.2 Les types de tests . . . 61
7.3 Construire un plan de test . . . 62
7.4 Scénarios et cas de test . . . 62
7.5 Automatiser : exemples de tests . . . 63
7.6 Identifier, corriger et suivre les bugs . . . 64
7.7 Documenter les résultats et valider . . . 65

Chapitre 08 — Phase 7 : déploiement et livraison . . . 67

8.1 Préparer l'environnement de production . . . 68
8.2 Choisir une solution d'hébergement . . . 68
8.3 Configurer le serveur . . . 69
8.4 Déployer le back-end, le front-end et la base . . . 70
8.5 Sécurité de production et sauvegardes . . . 70
8.6 Vérification finale et livrables . . . 71

Chapitre 09 — Documentation et rapport final . . . 72

9.1 Les documents à produire . . . 73
9.2 Documentation d'API et guide d'installation . . . 73
9.3 Guide utilisateur, captures et diagrammes . . . 74
9.4 Relier les phases au rapport de PFE . . . 74
9.5 Constituer le dossier documentaire . . . 75

Chapitre 10 — Gestion du projet et conseils professionnels . . . 76

10.1 Organiser son temps . . . 77
10.2 Planning global d'un PFE en cascade . . . 77
10.3 Travailler avec un encadrant . . . 78
10.4 Gérer les imprévus et les changements . . . 78
10.5 Suivre l'avancement et documenter les décisions . . . 79
10.6 Présenter les livrables et la démonstration finale . . . 80

10.7 Les 20 erreurs à éviter . . . 80
10.8 Se démarquer en tant que futur ingénieur logiciel . . . 81

Section spéciale — Douze prompts IA pour les phases du projet . . . 82

Boîte à outils — Boîte à outils : douze modèles réutilisables . . . 84

Modèle 1 — Modèle de document d'analyse des besoins . . . 84
Modèle 2 — Modèle de cahier des charges . . . 85
Modèle 3 — Modèle de spécifications fonctionnelles . . . 85
Modèle 4 — Modèle de spécifications techniques . . . 86
Modèle 5 — Modèle de planning prévisionnel . . . 86
Modèle 6 — Exemple de diagramme de cas d'utilisation . . . 87
Modèle 7 — Exemple de diagramme de classes . . . 87
Modèle 8 — Exemple de diagramme de séquence . . . 88
Modèle 9 — Modèle de plan de test . . . 88
Modèle 10 — Modèle de documentation API . . . 89
Modèle 11 — Modèle de README GitHub . . . 89
Modèle 12 — Checklist de livraison du projet . . . 90

Annexes — Annexes et bibliographie . . . 91

A Glossaire . . . 91
B Bloc de style PlantUML utilisé dans ce guide . . . 91
C Bibliographie et ressources . . . 92

M.D.S

CHAPITRE 01

Comprendre la méthode en cascade

01

Définition, origine, principes, forces et limites du modèle Waterfall, et critères pour décider s'il convient à votre PFE.

AU PROGRAMME

1.1 Définition et principe de fonctionnement

1.2 Historique et origine de la méthode

1.3 Pourquoi l'utiliser dans un PFE ?

1.4 Caractéristiques, avantages, inconvénients et limites

1.5 Waterfall et Agile / Scrum : comparaison

1.6 Dans quels contextes choisir la cascade ?

LIVRABLE ET OUTILS ASSOCIÉS

Schéma du cycle de développement · Critères de choix de la méthode

1.1 Définition et principe de fonctionnement

Aa **DÉFINITION**

Modèle en cascade (Waterfall Model)

Modèle de cycle de vie du logiciel dans lequel le projet est découpé en **phases séquentielles** (analyse, spécification, conception, réalisation, tests, livraison). Chaque phase produit des **livrables documentés** qui sont **validés** avant de passer à la phase suivante.

L'image de la cascade vient de l'enchaînement « de haut en bas » : le résultat d'une phase alimente la suivante, comme l'eau qui tombe d'un palier à l'autre. Le travail est donc planifié **en amont** et on cherche à prendre les bonnes décisions le plus tôt possible, car modifier un choix de conception tard dans le projet coûte plus cher.

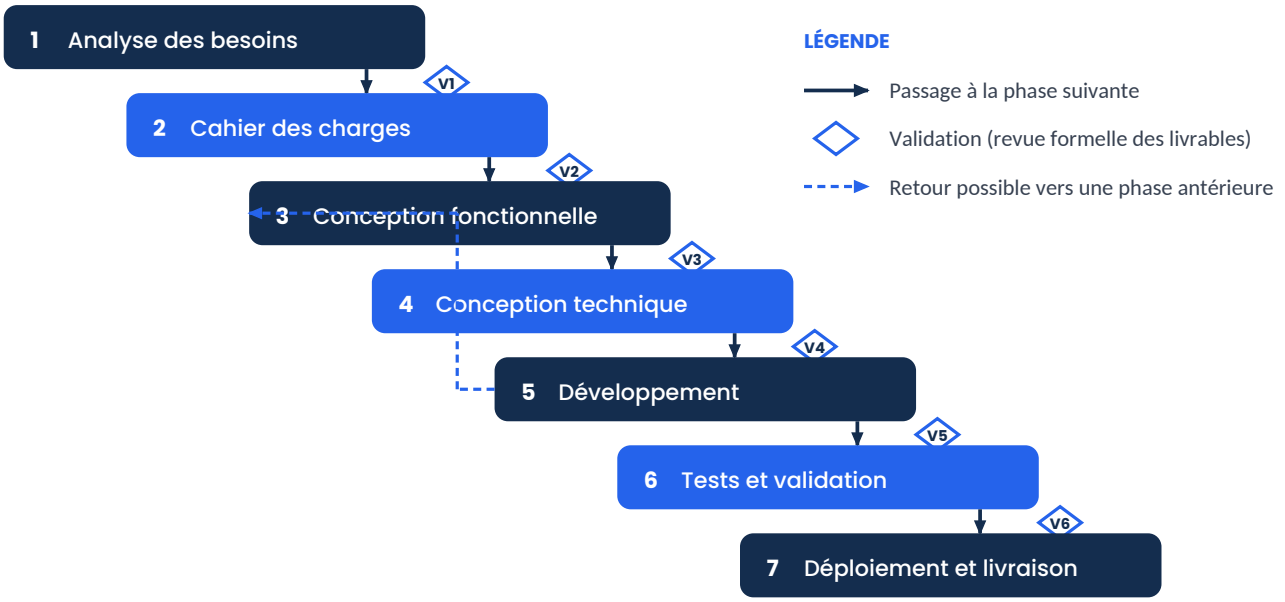


Figure 1 — Le cycle en cascade appliqué à un PFE : sept phases et six validations intermédiaires.

Séquentiel ne veut pas dire sans retour

Les phases sont **organisées séquentiellement** et les validations (V1, V2…) interviennent **avant** le passage à l'étape suivante : on ne commence pas à coder tant que la conception n'est pas validée. Mais les **retours en arrière restent possibles** : si un test révèle une erreur de conception, ou si le client change un besoin, on revient à la phase concernée, on met à jour les documents et on revalide. La différence avec une approche itérative est que ce retour est un **événement exceptionnel et contrôlé** (demande de modification), pas le mode de fonctionnement normal.

Phase	Question à laquelle elle répond	Livrable
1. Analyse des besoins	Quel problème résoudre, pour qui ?	Document d'analyse des besoins
2. Cahier des charges	Que doit faire exactement le système ?	Cahier des charges et spécifications
3. Conception fonctionnelle	Comment le système se comporte-t-il du point de vue métier ?	Diagrammes UML, spécifications fonctionnelles
4. Conception technique	Avec quelle architecture et quelles technologies ?	Dossier de conception technique

Phase	Question à laquelle elle répond	Livrable
5. Développement	Comment le construire ?	Code source, base de données
6. Tests et validation	Le système respecte-t-il les exigences ?	Rapport de tests
7. Déploiement et livraison	Comment le mettre à disposition ?	Application déployée, documentation

La maintenance (corriger et faire évoluer le logiciel après la livraison) est souvent représentée comme une huitième phase. Dans un PFE, elle se limite généralement à des perspectives d'évolution.

1.2 Historique et origine de la méthode

Le modèle séquentiel par phases est généralement rattaché à l'article de **Winston W. Royce**, « Managing the Development of Large Software Systems » (1970). Royce y présente un enchaînement de phases (exigences, analyse, conception, codage, tests, exploitation) comme point de départ, puis explique qu'une application stricte de ce schéma est **risquée** pour les projets importants et recommande des boucles de retour, de la documentation et l'implication du client. Le terme « waterfall » a été utilisé plus tard dans la littérature (on le cite souvent à propos de travaux des années 1970) et s'est imposé pour désigner ce modèle.

> **À SAVOIR**
> **Une nuance importante**
> On présente parfois Royce comme l'« inventeur » d'une méthode rigide. En réalité, son texte décrit le schéma séquentiel pour en montrer les limites. Retenez surtout que la cascade a structuré pendant des décennies les projets de grande taille, notamment ceux qui exigent des documents formels et des contrats précis.

Le modèle a donné naissance à des variantes, dont le **modèle en V**, qui met chaque phase de conception en regard d'une phase de test (exigences ↔ tests d'acceptation, conception ↔ tests d'intégration, code ↔ tests unitaires). Cette idée est précieuse en PFE : **on prépare les tests dès la conception** (chapitre 7).

1.3 Pourquoi l'utiliser dans un PFE ?

Le choix de la cascade en PFE s'explique rarement par la mode : il découle du contexte académique et du type de projet.

Raison	Explication
Livrables documentaires attendus	Le jury évalue souvent un cahier des charges, des diagrammes UML, un dossier de conception et un rapport : la cascade produit naturellement ces documents.
Besoins relativement stables	Le sujet est fixé au départ par l'établissement et l'entreprise ; le périmètre change peu.
Durée courte et jalons fixes	Avec quatre à six mois et des dates imposées, planifier toutes les phases dès le début rassure et facilite le suivi.
Progression pédagogique	Elle apprend à analyser avant de coder, à justifier ses choix et à valider.
Travail en solo ou binôme	Les rituels agiles d'équipe (daily, rétrospective) ont moins de sens ; des phases claires structurent mieux le travail individuel.

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

La méthode peut être imposée

Certains établissements ou entreprises imposent la cascade, d'autres une méthode agile, d'autres laissent le choix. Vérifiez la consigne avant de vous engager, et justifiez votre choix dans le rapport.

1.4 Caractéristiques, avantages, inconvénients et limites

Les caractéristiques principales

- **Phases séquentielles** avec entrées et sorties précises.
- **Documentation abondante** : chaque phase est formalisée par un livrable.
- **Validations (jalons)** avant de passer à la phase suivante.
- **Planification globale** en début de projet (planning, charge, risques).
- **Traçabilité** : chaque exigence est reliée à la conception, au code et aux tests.
- **Changement coûteux** lorsqu'il intervient tard.

Avantages	Inconvénients et limites
• Simple à comprendre et à expliquer. • Planning et jalons clairs, faciles à suivre. • Documentation complète, utile pour le rapport et pour la maintenance. • Responsabilités et livrables bien définis. • Bien adapté aux exigences stables et aux contextes contractuels.	• Le client voit le produit tard : les malentendus apparaissent tard. • Les erreurs d'analyse sont découvertes pendant les tests, donc coûteuses. • Peu flexible face à un besoin qui évolue. • Risque de « tunnel » : beaucoup de documents avant d'avoir du code qui fonctionne. • Les tests ont lieu tard si on n'anticipe pas leur préparation.

ATTENTION

Le piège de la cascade « de façade »

Appliquer la cascade en rédigeant des documents que personne ne relit, ou en validant sans réelle revue, ne protège de rien. Une validation est un acte réel : relecture, questions, corrections, puis accord écrit (compte rendu ou signature).

1.5 Waterfall et Agile / Scrum : comparaison

DÉFINITION

Agile et Scrum

Les méthodes agiles (Manifeste agile, 2001) privilégient la livraison fréquente de petits incréments et l'adaptation au changement. **Scrum**, décrit dans le Scrum Guide, organise le travail en sprints d'un mois maximum avec des rôles et des événements précis.

Critère	Cascade (Waterfall)	Agile / Scrum
Organisation	Phases séquentielles.	Itérations courtes (sprints).
Planification	Globale, en début de projet.	Progressive, révisée à chaque itération.
Changement de besoin	Possible mais encadré et coûteux.	Attendu et intégré au backlog.
Première version utilisable	En fin de projet.	Dès les premières itérations.
Documentation	Abondante, formelle.	Suffisante, plus légère.
Implication du client	Aux validations de phase.	Continue.
Gestion des risques	Anticipée par l'analyse initiale.	Réduite par les retours fréquents.
Adaptation à un PFE	Très bonne si besoin stable et livrables documentaires.	Très bonne si besoin évolutif et dialogue régulier avec l'entreprise.

CONSEIL PROFESSIONNEL

Une approche hybride est possible

Beaucoup de PFE combinent une phase initiale en cascade (analyse, cahier des charges, conception validés) avec un développement par petites itérations internes. Si vous le faites, dites-le clairement dans le rapport et justifiez-le.

1.6 Dans quels contextes choisir la cascade ?

Choisir la cascade si…	Préférer l'agile si…
• Les exigences sont connues et stables. • Le périmètre est borné et validé par écrit. • Le livrable est évalué sur la qualité des documents. • Le client est peu disponible entre les jalons. • Le système est critique ou réglementé et exige une traçabilité forte.	• Le besoin est flou et va évoluer. • Le client peut tester régulièrement. • Une version utilisable rapidement est un enjeu. • La technologie est nouvelle pour vous et les risques sont élevés. • L'entreprise travaille déjà en sprints et attend de vous la même méthode.

Ex

EXEMPLE PRATIQUE

SupportIT : pourquoi la cascade ?

Atlas Services (fictive) demande une application dont les fonctions sont bien identifiées (tickets, affectation, statuts, tableau de bord), à livrer à une date fixe, avec un cahier des charges et des diagrammes UML exigés par le jury. Le besoin est stable, le client est disponible pour des validations mensuelles : la cascade est un choix cohérent.

À RETENIR

Ce que vous retenez

La cascade enchaîne des phases validées, chacune produisant des livrables documentés ; les retours en arrière restent possibles mais contrôlés.

Elle convient aux besoins stables et aux projets évalués sur leurs documents.

Sa force est la traçabilité ; sa faiblesse, la découverte tardive des erreurs : préparez les tests dès la conception.

CHAPITRE 02

Phase 1 : étude préliminaire et analyse des besoins

02

Démarrer concrètement : comprendre le problème et le contexte, recueillir les besoins, fixer objectifs et périmètre, étudier la faisabilité.

AU PROGRAMME

2.1 Identifier le problème et comprendre le contexte

2.2 Identifier les utilisateurs et les parties prenantes

2.3 Recueillir les besoins

2.4 Organiser les réunions avec le client ou l'encadrant

2.5 Définir objectifs et périmètre

2.6 Contraintes et faisabilité

2.7 Besoins fonctionnels et non fonctionnels : SupportIT

2.8 Personas et acteurs

2.9 Livrable : le document d'analyse des besoins

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : document d'analyse des besoins (validation V1) · Toolbox : modèle n°1

2.1 Identifier le problème et comprendre le contexte

Un projet commence par un problème, pas par une technologie. Cette phase consiste à comprendre ce qui ne va pas, pour qui, et pourquoi cela vaut la peine d'être résolu.

Partir d'une situation réelle

Interrogez les personnes concernées et observez le fonctionnement actuel. Posez-vous cinq questions : **qui** est gêné, **quoi** exactement, **quand** et **où** cela arrive, et **quelles conséquences** cela a. La technique des « 5 pourquoi » aide à remonter à la cause réelle plutôt qu'au symptôme.

Ex

EXEMPLE PRATIQUE

SupportIT — le contexte

Entreprise (fictive) : Atlas Services, société de services d'environ 150 employés, avec un service informatique de quatre personnes.

Situation actuelle : les employés signalent leurs pannes par téléphone, par e-mail ou en passant au bureau. Les demandes ne sont ni centralisées ni numérotées.

Conséquences : demandes oubliées, priorités mal comprises, impossibilité de savoir où en est une intervention, aucun historique ni statistique.

Ces éléments sont des données pédagogiques fictives.

Ex

EXEMPLE PRATIQUE

SupportIT — exemple de problématique

« Comment centraliser les demandes d'assistance informatique d'Atlas Services et permettre le suivi de leur traitement, afin de réduire les oublis et d'offrir une visibilité sur l'activité du service informatique ? »

Comprendre le contexte de l'entreprise

À comprendre	Questions concrètes
Activité et organisation	Que fait l'entreprise ? Quels services ? Qui décide ?
Processus actuel	Comment une demande est-elle traitée aujourd'hui, étape par étape ?
Outils existants	Quels logiciels, fichiers, messageries sont utilisés ? Peuvent-ils être réutilisés ?
Environnement technique	Postes, navigateurs, réseau, serveurs, règles de sécurité.
Enjeux	Gain de temps, qualité de service, coûts, conformité.

2.2 Identifier les utilisateurs et les parties prenantes

Aa **DÉFINITION**

Partie prenante

Toute personne ou entité qui utilise le système, le finance, le valide ou subit ses effets : utilisateurs, commanditaire, encadrants, service sécurité, direction...

Partie prenante	Rôle dans SupportIT	Attentes
Employé	Signale un incident et suit sa résolution.	Simplicité, rapidité, visibilité sur l'avancement.
Technicien informatique	Traite les interventions.	Liste claire de ses tickets, historique, outils de communication.
Administrateur (responsable IT)	Gère les comptes, affecte, pilote l'activité.	Vue d'ensemble, statistiques, contrôle des accès.
Direction	Commanditaire (fictif).	Qualité de service, indicateurs, coût maîtrisé.
Encadrant professionnel / académique	Valident les livrables.	Rigueur, conformité aux exigences, calendrier tenu.

2.3 Recueillir les besoins

Le recueil des besoins (ou **élicitation**) combine plusieurs techniques. Aucune ne suffit seule.

Technique	Principe	Avantage	Limite
Entretien	Échange individuel structuré.	Détail et nuances.	Chronophage.
Observation	Regarder le travail réel.	Révèle ce que personne ne dit.	Demande de la disponibilité.
Questionnaire	Questions standardisées.	Beaucoup de réponses rapidement.	Peu de profondeur.
Analyse documentaire	Lire procédures, formulaires, anciens fichiers.	Objectif, rapide.	Peut être obsolète.
Atelier	Réunion de groupe sur un thème.	Compromis rapide entre points de vue.	Nécessite une bonne animation.
Maquette / prototype	Montrer des écrans.	Fait réagir, évite les malentendus.	Peut figer trop tôt.

Exemple de questions pour un entretien (SupportIT)

- « Décrivez la dernière fois où vous avez eu une panne : que s'est-il passé, étape par étape ? »
- « Qu'aimeriez-vous savoir sur l'avancement de votre demande ? »
- « Quelles informations vous manquent le plus quand vous traitez une demande ? »
- « Quels indicateurs vous seraient utiles pour piloter le service ? »

- « Quelles règles de sécurité doivent absolument être respectées ? »

CONSEIL PROFESSIONNEL

Reformuler et valider

Après chaque entretien, reformulez les besoins par écrit et faites-les confirmer. Une phrase comme « Si j'ai bien compris… » évite les mauvaises surprises en phase de validation.

2.4 Organiser les réunions avec le client ou l'encadrant

Moment	Objectif	Préparation
Réunion de lancement	Comprendre le sujet, présenter la démarche, fixer le rythme.	Liste de questions, proposition de planning.
Ateliers de recueil	Collecter besoins et règles de gestion.	Guide d'entretien, supports visuels.
Revue d'analyse (V1)	Valider le document d'analyse.	Document envoyé à l'avance, points ouverts listés.

COMPTE RENDU DE RÉUNION — EXEMPLE (cas fictif SupportIT)

Réunion : Cadrage du besoin — **Participants :** [Responsable IT], [Encadrant], [Étudiant] — **Durée :** 1 h

Points abordés : processus actuel de traitement des demandes ; problèmes rencontrés ; rôles utilisateurs attendus.

Décisions : trois rôles (employé, technicien, administrateur) ; cinq statuts de ticket ; pas d'application mobile dans cette version.

Actions : fournir un exemple de demandes récentes (Responsable IT, S+1) ; rédiger le document d'analyse (Étudiant, S+2).

Prochaine réunion : revue du document d'analyse.

2.5 Définir objectifs et périmètre

Objectifs généraux et spécifiques

DÉFINITION

Objectif général / spécifique

L'**objectif général** exprime le but global du projet. Les **objectifs spécifiques** le décomposent en résultats précis, mesurables et réalisables dans le temps imparti (critères SMART).

Ex

EXEMPLE PRATIQUE

SupportIT — objectifs

Général : mettre à disposition d'Atlas Services une application web centralisant les demandes d'assistance et leur traitement.

OS1 : permettre à un employé de créer et suivre ses tickets.

OS2 : permettre à l'administrateur d'affecter chaque ticket à un technicien.

OS3 : tracer chaque changement de statut avec son historique.

OS4 : offrir un tableau de bord avec des statistiques de base.

OS5 : sécuriser l'accès par authentification et rôles.

Délimiter le périmètre

Inclus	Exclus (hors périmètre)
• Authentification et gestion des utilisateurs	• Application mobile native
• Création, affectation, suivi et clôture des tickets	• Intégration avec l'annuaire d'entreprise
• Commentaires et historique	• Notifications par SMS
• Tableau de bord et statistiques	• Gestion du parc matériel (inventaire)

Faites valider le périmètre par écrit : c'est votre meilleur outil pour gérer les demandes de dernière minute.

2.6 Contraintes et faisabilité

Les **contraintes** limitent les choix possibles ; la **faisabilité** vérifie que le projet peut être mené à bien avec les moyens disponibles.

Type de contrainte	Exemples pour SupportIT
Fonctionnelles	Trois rôles ; cinq statuts ; historique des changements conservé.
Techniques	Application web accessible depuis un navigateur récent ; base relationnelle MySQL.
Organisationnelles	Un étudiant ; validations mensuelles avec le responsable IT.
Temporelles	Durée du stage ; dates de dépôt et de soutenance de l'établissement.
Sécurité	Mots de passe hachés ; accès par rôles ; HTTPS en production.

Dimension de faisabilité	Question	Conclusion SupportIT (fictive)
Technique	Les technologies sont-elles maîtrisées ?	Oui : stack React, Node.js, Express, MySQL connue.
Organisationnelle	Les utilisateurs sont-ils disponibles ?	Oui, pour des validations régulières.
Temporelle	Le périmètre tient-il dans le planning ?	Oui, après exclusion de l'application mobile.
Économique	Les coûts d'outils et d'hébergement sont-ils acceptables ?	Oui : outils libres, hébergement étudiant.
Juridique / sécurité	Données personnelles et règles internes respectées ?	À valider : minimisation des données, accès contrôlé.

2.7 Besoins fonctionnels et non fonctionnels : SupportIT

Aa

DÉFINITION

Besoin fonctionnel / non fonctionnel

Un **besoin fonctionnel** décrit ce que le système doit faire. Un **besoin non fonctionnel** décrit comment il doit le faire : performance, sécurité, ergonomie, disponibilité, maintenabilité…

ID	Besoin fonctionnel	Acteur
BF-01	S'authentifier avec e-mail et mot de passe.	Tous
BF-02	Créer, modifier, désactiver un compte utilisateur.	Administrateur
BF-03	Créer un ticket (titre, description, catégorie, priorité).	Employé
BF-04	Consulter la liste et le détail de ses tickets.	Employé
BF-05	Affecter un ticket à un technicien.	Administrateur
BF-06	Changer le statut d'un ticket affecté.	Technicien
BF-07	Ajouter un commentaire à un ticket.	Tous (selon droits)
BF-08	Consulter l'historique des statuts d'un ticket.	Tous (selon droits)
BF-09	Consulter un tableau de bord.	Administrateur
BF-10	Consulter des statistiques (tickets par statut, par catégorie, délai moyen).	Administrateur
BF-11	Filtrer et rechercher des tickets.	Tous (selon droits)

ID	Besoin non fonctionnel	Critère vérifiable
BNF-01	Sécurité des mots de passe.	Stockage sous forme hachée (bcrypt) ; jamais en clair.
BNF-02	Contrôle d'accès.	Chaque route protégée vérifie jeton et rôle.
BNF-03	Performance.	Liste de tickets affichée en moins de 2 secondes pour 5 000 tickets de test (mesuré en recette).
BNF-04	Ergonomie.	Créer un ticket en moins de 5 champs obligatoires.
BNF-05	Compatibilité.	Navigateurs récents (versions actuelles).
BNF-06	Maintenabilité.	Code organisé en couches, README et documentation d'API.
BNF-07	Traçabilité.	Tout changement de statut enregistré avec date et auteur.

ATTENTION

Un besoin doit être vérifiable

« L'application doit être rapide » n'est pas exploitable. Écrivez une valeur, une condition de mesure, ou supprimez le besoin. Les valeurs ci-dessus sont des exemples pédagogiques à adapter.

2.8 Personas et acteurs

DÉFINITION

Acteur et persona

Un **acteur** est un rôle qui interagit avec le système (employé, technicien, administrateur). Un **persona** est un utilisateur type fictif, décrit avec ses objectifs et ses difficultés, pour concevoir en pensant à de vraies personnes.

Persona (fictif)	Acteur	Contexte et objectifs	Difficultés actuelles
Salma, comptable	Employé	Signale vite une panne d'imprimante pour reprendre son travail.	Ne sait pas à qui s'adresser ni où en est sa demande.
Yassine, technicien support	Technicien	Traite plusieurs interventions par jour, veut prioriser.	Demandes dispersées entre appels et e-mails.
Nadia, responsable informatique	Administrateur	Gère l'équipe et rend compte à la direction.	Aucune vision chiffrée de l'activité.

2.9 Livrable : le document d'analyse des besoins

Le document d'analyse des besoins rassemble les résultats de la phase 1. Il est **validé (V1)** par l'encadrant ou le client avant de passer au cahier des charges.

Section	Contenu
1. Introduction	Objet du document, périmètre, lecteurs, historique des versions.
2. Contexte	Entreprise, situation actuelle, enjeux.
3. Problématique	Problème et question centrale.
4. Parties prenantes et acteurs	Liste, rôles, personas.
5. Recueil des besoins	Méthodes utilisées, comptes rendus (en annexe).
6. Objectifs et périmètre	Généraux, spécifiques, inclus / exclu.
7. Besoins	Fonctionnels et non fonctionnels, avec identifiants.
8. Contraintes et faisabilité	Analyse par dimension.
9. Risques	Principaux risques et premières mesures.
10. Validation	Nom, date et signature des validateurs.

À RETENIR

Ce que vous retenez

Comprenez le problème avant de proposer une solution.

Combinez plusieurs techniques de recueil et faites valider vos reformulations.

Numérotez chaque besoin (BF, BNF) : ces identifiants serviront à la traçabilité jusqu'aux tests.

M.D.S

CHAPITRE 03

Phase 2 : cahier des charges et spécifications

03

Transformer les besoins en exigences précises, structurer un cahier des charges professionnel et organiser la traçabilité.

AU PROGRAMME

3.1 Besoin, exigence, fonctionnalité, contrainte, critère d'acceptation

3.2 Structure d'un cahier des charges professionnel

3.3 Rédiger de bonnes exigences

3.4 Prioriser et tracer les exigences

3.5 Valider le cahier des charges

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : cahier des charges validé (V2) · Toolbox : modèles n°2 et n°3

3.1 Besoin, exigence, fonctionnalité, contrainte, critère d'acceptation

Ces cinq termes sont souvent confondus. Les distinguer rend vos documents plus clairs et vos validations plus faciles.

Terme	Définition	Exemple SupportIT
Besoin	Ce que l'utilisateur ou l'entreprise veut obtenir, exprimé de son point de vue.	Un employé veut signaler une panne et savoir où en est sa demande.
Exigence	Besoin reformulé de façon précise, vérifiable et numérotée, que le système doit satisfaire.	EX-03 : le système doit permettre à un employé authentifié de créer un ticket avec titre, description, catégorie et priorité.
Fonctionnalité	Service concret fourni par le système pour répondre à une exigence.	Formulaire de création de ticket.
Contrainte	Limite imposée au projet (technique, organisationnelle, réglementaire, temporelle).	Base de données MySQL ; livraison avant la date de dépôt du rapport.
Critère d'acceptation	Condition qui permet de dire que l'exigence est satisfaite.	Après validation du formulaire, un ticket au statut OUVERT apparaît dans la liste de l'employé.



À RETENIR

La chaîne logique

Besoin → exigence → fonctionnalité → critère d'acceptation → cas de test. Si un maillon manque, la validation devient subjective.

3.2 Structure d'un cahier des charges professionnel

Voici les douze sections attendues, avec un extrait illustré par SupportIT. Le modèle complet à adapter se trouve dans la boîte à outils.

Section	Contenu attendu	Extrait SupportIT (fictif)
1. Présentation générale	Projet, auteur, version, destinataires.	SupportIT — version 1.0 — destiné à Atlas Services.
2. Contexte du projet	Entreprise, situation, enjeux.	Demandes d'assistance dispersées entre téléphone et e-mails.
3. Problématique	Problème et question centrale.	Comment centraliser et suivre les demandes d'assistance ?
4. Objectifs	Généraux et spécifiques.	OS1 à OS5 (voir chapitre 2).
5. Étude de l'existant	Processus et outils actuels, limites, solutions du marché étudiées.	E-mails et appels ; aucun historique ; solutions existantes jugées trop lourdes (hypothèse fictive).
6. Solution proposée	Description globale du système.	Application web avec trois rôles et cycle de vie de ticket.
7. Besoins fonctionnels	Liste numérotée priorisée.	EX-01 à EX-11.

Section	Contenu attendu	Extrait SupportIT (fictif)
8. Besoins non fonctionnels	Qualités attendues, mesurables.	Sécurité, performance, ergonomie…
9. Contraintes techniques	Technologies, environnement, sécurité.	React, Node.js/Express, MySQL, JWT.
10. Périmètre	Inclus / exclu.	Hors périmètre : application mobile.
11. Planning prévisionnel	Phases, jalons, livrables.	Gantt sur 24 semaines (chapitre 10).
12. Critères d'acceptation	Conditions de validation de chaque exigence.	Voir tableau de traçabilité.

3.3 Rédiger de bonnes exigences

Une bonne exigence est…	À éviter
• **Unique** : une seule idée par exigence. • **Précise** : sans adjectifs vagues. • **Vérifiable** : on peut la tester. • **Réaliste** dans le temps et les moyens. • **Numérotée** et versionnée. • **Formulée avec « doit »** pour une exigence obligatoire.	• « Le système doit être convivial. » • « L'application doit gérer les tickets » (trop large). • Mélanger besoin et solution technique dans la même phrase. • Exigences contradictoires non signalées. • Termes non définis (glossaire absent).

Exigence faible	Exigence améliorée
Le système permet de gérer les tickets.	EX-06 : un technicien doit pouvoir faire passer un ticket qui lui est affecté d'un statut vers le suivant autorisé (voir diagramme d'états).
L'application est sécurisée.	EX-12 : toute route de l'API autre que la connexion doit refuser (code 401) une requête sans jeton valide.
Les pages sont rapides.	EX-13 : la liste de 5 000 tickets de test doit s'afficher en moins de 2 secondes lors de la recette.

3.4 Prioriser et tracer les exigences

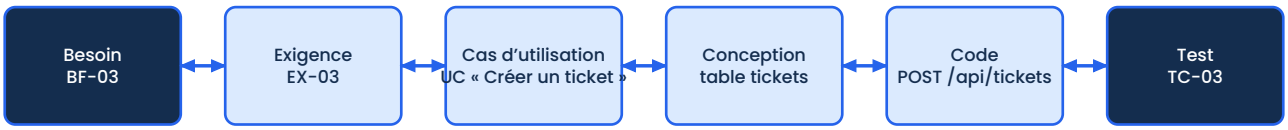
La méthode MoSCoW classe les exigences en **Must** (indispensable), **Should** (important), **Could** (souhaitable) et **Won't (now)** (exclu pour cette version). Elle protège votre planning : si le temps manque, on sacrifie d'abord les « Could ».

Aa

DÉFINITION

Traçabilité des exigences

Capacité de suivre chaque exigence depuis son origine (le besoin) jusqu'à sa conception, son implémentation et son test, et inversement. Elle est présentée sous forme de **matrice de traçabilité**.



Chaque besoin doit pouvoir être suivi jusqu'au test qui le valide, et inversement.

Figure 2 — La chaîne de traçabilité d'une exigence.

Besoin	Exigence	Priorité	Cas d'utilisation	Conception	Test
BF-01	EX-01	Must	UC S'authentifier	Table utilisateurs	TC-01, TC-02
BF-03	EX-03	Must	UC Créer un ticket	Table tickets	TC-03, TC-04
BF-05	EX-05	Must	UC Affecter un ticket	Champ technicien_id	TC-05
BF-06	EX-06	Must	UC Changer le statut	Diagramme d'états	TC-06, TC-07
BF-09	EX-09	Should	UC Consulter le tableau de bord	Requêtes agrégées	TC-09
BF-10	EX-10	Could	UC Statistiques avancées	Vues SQL	TC-10

CONSEIL PROFESSIONNEL

Maintenez la matrice vivante

Un tableur suffit. Mettez-la à jour à chaque modification d'exigence : elle vous permettra de répondre au jury « où cette exigence est-elle testée ? » en quelques secondes.

3.5 Valider le cahier des charges

La validation V2 est un jalon essentiel : après elle, tout changement passe par une **demande de modification** (chapitre 10).

1. **Relecture croisée**
   L'encadrant relit le document : cohérence, clarté, complétude.
2. **Réunion de validation**
   Examen des points ouverts, décisions consignées dans un compte rendu.
3. **Corrections**
   Nouvelle version (V1.1) avec historique des modifications.
4. **Accord écrit**
   Signature, e-mail de validation ou procès-verbal : une trace datée est indispensable.
5. **Gel des exigences**
   Les exigences validées sont « gelées » ; les évolutions suivent la procédure de changement.

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Formalisme de validation

Certains établissements exigent un document signé ou une fiche de validation ; d'autres acceptent un échange écrit. Demandez ce qui est attendu.

À RETENIR

Ce que vous retenez

Distinguez besoin, exigence, fonctionnalité, contrainte et critère d'acceptation.

Une exigence est unique, précise, vérifiable et numérotée.

La matrice de traçabilité relie chaque exigence à sa conception et à son test.

M.D.S

CHAPITRE 04

Phase 3 : conception fonctionnelle et UML

04

Modéliser le système du point de vue métier : cas d'utilisation, classes, séquences, activités et états, avec le code PlantUML pour reproduire chaque diagramme.

AU PROGRAMME

4.1 UML : pourquoi et comment l'utiliser

4.2 Diagramme de cas d'utilisation

4.3 Spécifications fonctionnelles détaillées

4.4 Diagramme de classes

4.5 Diagramme de séquence

4.6 Diagramme d'activité

4.7 Diagramme d'états-transitions

4.8 Cohérence entre les diagrammes

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : dossier de conception fonctionnelle (validation V3) · Toolbox : modèles n°3, 6, 7, 8

4.1 UML : pourquoi et comment l'utiliser

Aa

DÉFINITION

UML (Unified Modeling Language)

Langage de modélisation graphique normalisé (standard de l'OMG) qui permet de représenter un système logiciel sous plusieurs angles : fonctionnel, structurel, comportemental. UML 2 définit 14 types de diagrammes ; un PFE en utilise en pratique quatre à six.

Dans la cascade, les diagrammes UML sont des **livrables de conception** : ils sont relus, validés et servent de contrat entre l'analyse et le développement. Chaque diagramme répond à une question précise :

Diagramme	Question	Point de vue	Phase
Cas d'utilisation	Qui fait quoi avec le système ?	Besoins	Spécification
Classes	Quelles données et quelles relations ?	Structure	Conception
Séquence	Comment les éléments collaborent-ils dans le temps ?	Comportement	Conception
Activité	Quel est l'enchaînement d'un processus ?	Comportement	Spécification / conception
États-transitions	Quels sont les états d'un objet et comment change-t-il ?	Comportement	Conception

Comment reproduire les diagrammes avec PlantUML

PlantUML est un outil libre qui génère des diagrammes à partir d'un texte. Avantages : le code se versionne avec Git, se relit facilement et se corrige en quelques secondes. Tous les diagrammes de ce guide sont produits ainsi ; vous pouvez coller le code dans un éditeur PlantUML en ligne ou dans une extension de votre éditeur (par exemple VS Code) pour obtenir le même résultat.

STRUCTURE DE BASE D'UN FICHIER PLANTUML

```
@startuml
' Mon premier diagramme
actor Employé
Employé -> Système : crée un ticket
@enduml
```

CONSEIL PROFESSIONNEL

Version texte et image

Conservez dans votre dépôt le fichier .puml **et** l'image exportée (PNG ou SVG). Dans le rapport, insérez l'image ; dans les annexes, donnez le code. Les couleurs de ce guide sont obtenues avec un petit bloc de style « skinparam » commun, facultatif pour vos propres diagrammes.

4.2 Diagramme de cas d'utilisation

Aa **DÉFINITION**

Diagramme de cas d'utilisation

Représentation des **acteurs** (rôles externes) et des **cas d'utilisation** (objectifs qu'ils atteignent avec le système), délimités par la frontière du système.

Utilité

C'est le diagramme de dialogue avec le client : lisible sans connaissance technique, il valide le périmètre fonctionnel. Il sert de base aux tests d'acceptation.

Composants

Élément	Notation	Remarque
Acteur	Silhouette (bonhomme)	Rôle, pas une personne : « Employé », pas « Salma ».
Cas d'utilisation	Ellipse	Verbe à l'infinitif + complément.
Système	Rectangle	Frontière : ce qui est dedans fait partie du projet.
Association	Trait plein	Acteur participe au cas.
« include »	Flèche pointillée	Le cas de départ inclut toujours le cas d'arrivée.
« extend »	Flèche pointillée	Comportement optionnel ajouté à un cas de base.

Comment le construire

1. Listez les acteurs (qui utilise le système ?).
2. Pour chacun, listez ses objectifs (que veut-il accomplir ?).
3. Nommez chaque objectif par un verbe.
4. Regroupez, évitez les doublons, cherchez les cas inclus ou étendus.
5. Dessinez la frontière du système et reliez acteurs et cas.
6. Faites relire par le client.

Exemple complet : SupportIT

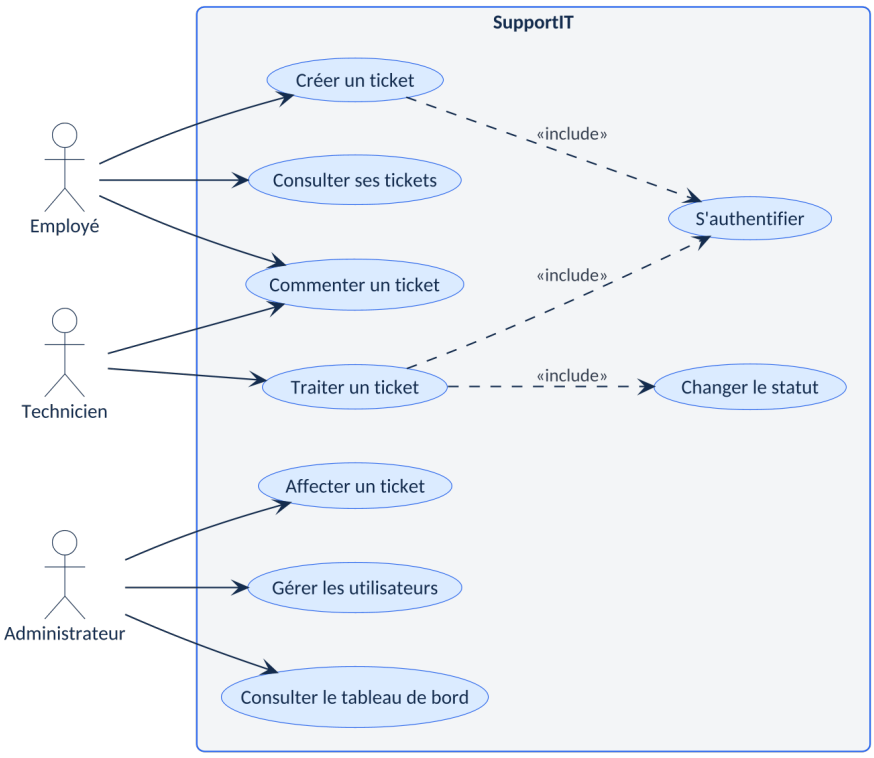


Figure 3 — Cas d'utilisation de SupportIT (le cas « S'authentifier » est inclus par tous les cas de l'application ; seuls deux liens sont dessinés pour la lisibilité).

PLANTUML — CAS D'UTILISATION

```
@startuml
left to right direction
actor "Employé" as EMP
actor "Technicien" as TEC
actor "Administrateur" as ADM
rectangle "SupportIT" {
  usecase "S'authentifier" as UC1
  usecase "Créer un ticket" as UC2
  usecase "Consulter ses tickets" as UC3
  usecase "Commenter un ticket" as UC4
  usecase "Traiter un ticket" as UC5
  usecase "Changer le statut" as UC6
  usecase "Affecter un ticket" as UC7
  usecase "Gérer les utilisateurs" as UC8
  usecase "Consulter le tableau de bord" as UC9
}
EMP --> UC2
EMP --> UC3
EMP --> UC4
TEC --> UC5
TEC --> UC4
ADM --> UC7
ADM --> UC8
ADM --> UC9
UC2 ..> UC1 : <<include>>
UC5 ..> UC1 : <<include>>
UC5 ..> UC6 : <<include>>
@enduml
```

Erreurs fréquentes

- Décrire des **écrans** ou des actions techniques (« cliquer sur le bouton ») au lieu d'objectifs.
- Nommer des acteurs par des personnes ou des outils internes.
- Abuser des « include » / « extend » : si le doute persiste, supprimez-les.
- Oublier l'acteur qui administre le système.
- Cas d'utilisation trop larges (« Gérer le système »).

4.3 Spécifications fonctionnelles détaillées

Un cas d'utilisation sur un schéma ne suffit pas : on le **décrit textuellement** dans les spécifications fonctionnelles. Cette description sert à développer, tester et valider.

Rubrique	UC « Créer un ticket » (SupportIT)
Identifiant	UC-03 — lié à BF-03 / EX-03
Acteur principal	Employé
Préconditions	L'employé est authentifié.
Scénario nominal	• 1. L'employé ouvre le formulaire « Nouveau ticket ». • 2. Il saisit le titre, la description, la catégorie et la priorité. • 3. Il valide. • 4. Le système vérifie les champs. • 5. Le système enregistre le ticket au statut OUVERT et note la date. • 6. Le système affiche une confirmation et le ticket dans la liste de l'employé.
Scénarios alternatifs	• 4a. Un champ obligatoire est vide : le système affiche un message d'erreur et conserve la saisie. • 4b. Le jeton est expiré : le système redirige vers l'authentification.
Postconditions	Un ticket OUVERT existe ; une ligne d'historique est créée.
Règles de gestion	RG-01 : titre de 5 à 150 caractères. RG-02 : priorité par défaut NORMALE.

CONSEIL PROFESSIONNEL

Écrivez une fiche par cas important

Pas besoin de décrire les 9 cas avec le même niveau de détail : commencez par ceux qui portent la valeur du projet (créer, affecter, changer le statut). Un modèle réutilisable est fourni dans la boîte à outils.

4.4 Diagramme de classes

DÉFINITION

Diagramme de classes

Description de la **structure statique** du système : les classes (concepts), leurs attributs et opérations, et les relations entre elles (associations, compositions, héritage) avec leurs multiplicités.

Utilité

Il fixe le vocabulaire métier, prépare le modèle de données et le code. Au niveau de la conception fonctionnelle, on modélise les **concepts du domaine** avant les détails techniques.

Composants

Élément	Notation	Exemple
Classe	Rectangle à trois compartiments : nom, attributs, opérations.	Ticket
Attribut	visibilité nom : type	- titre : String
Opération	visibilité nom(params) : retour	+ affecter(t : Utilisateur) : void
Association	Trait avec multiplicités et nom	Utilisateur 1 — 0..* Ticket
Composition	Losange plein : le tout contrôle le cycle de vie des parties	Ticket — Commentaire
Énumération	Type avec valeurs fixes	Statut : OUVERT, AFFECTE…
Visibilité	+ public, - privé, # protégé	-

Comment le construire

1. Repérez les **noms** importants dans le cahier des charges : ce sont des classes candidates.
2. Pour chaque classe, listez ses attributs utiles (pas tous ceux imaginables).
3. Identifiez les associations et fixez les **multiplicités** (1, 0..1, *).
4. Repérez les compositions (le commentaire n'existe pas sans son ticket).
5. Ajoutez les énumérations (statuts, rôles, priorités).
6. Vérifiez que chaque exigence est couverte par au moins une classe.

Exemple complet : SupportIT

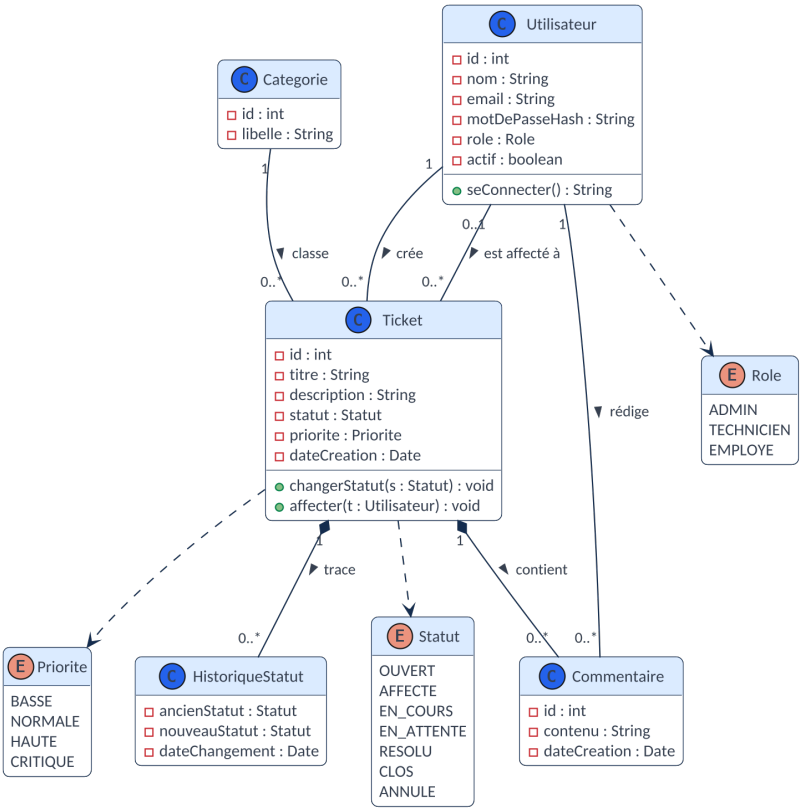


Figure 4 — Diagramme de classes de SupportIT.

PLANTUML — CLASSES

```
@startuml
hide empty members
enum Role {
  ADMIN
  TECHNICIEN
  EMPLOYE
}
enum Statut {
  OUVERT
  AFFECTE
  EN_COURS
  EN_ATTENTE
  RESOLU
  CLOS
  ANNULE
}
enum Priorite {
  BASSE
  NORMALE
  HAUTE
  CRITIQUE
}
class Utilisateur {
  - id : int
  - nom : String
  - email : String
  - motDePasseHash : String
  - role : Role
  - actif : boolean
  + seConnecter() : String
}
class Ticket {
  - id : int
  - titre : String
```

PLANTUML — CLASSES (suite)

```
  - description : String
  - statut : Statut
  - priorite : Priorite
  - dateCreation : Date
  + changerStatut(s : Statut) : void
  + affecter(t : Utilisateur) : void
}
class Categorie {
  - id : int
  - libelle : String
}
class Commentaire {
  - id : int
  - contenu : String
  - dateCreation : Date
}
class HistoriqueStatut {
  - ancienStatut : Statut
  - nouveauStatut : Statut
  - dateChangement : Date
}
Utilisateur "1" -- "0..*" Ticket : crée >
Utilisateur "0..1" -- "0..*" Ticket : est affecté à >
Categorie "1" -- "0..*" Ticket : classe >
Ticket "1" *-- "0..*" Commentaire : contient >
Utilisateur "1" -- "0..*" Commentaire : rédige >
Ticket "1" *-- "0..*" HistoriqueStatut : trace >
Utilisateur ..> Role
Ticket ..> Statut
Ticket ..> Priorite
@enduml
```

Erreurs fréquentes

- Mettre des **clés étrangères** (id_utilisateur) comme attributs : on modélise des associations, pas des colonnes.
- Multiplicités oubliées ou fausses.
- Classes « fourre-tout » (Gestionnaire, Manager) sans responsabilité claire.
- Confondre composition et association simple.
- Dupliquer la même information dans deux classes.

4.5 Diagramme de séquence

Aa

DÉFINITION

Diagramme de séquence

Représentation des **échanges de messages entre participants dans l'ordre chronologique** pour un scénario précis. Le temps s'écoule de haut en bas ; chaque participant a une ligne de vie verticale.

Utilité

Il clarifie la collaboration entre l'interface, l'API, les services et la base de données. Il se transforme presque directement en code et en cas de test, et il est très apprécié des jurys.

Composants

Élément	Description
Participant / acteur	Boîte en haut du diagramme : interface, API, service, base de données.
Ligne de vie	Trait vertical pointillé : existence du participant dans le temps.
Message synchrone	Flèche pleine : appel qui attend une réponse.
Message de retour	Flèche pointillée : réponse.
Fragment « alt »	Alternative : « si… sinon… » (chemins différents).
Fragment « loop » / « opt »	Boucle ou comportement optionnel.

Comment le construire

1. Choisissez **un scénario** précis (celui d'un cas d'utilisation).
2. Listez les participants dans l'ordre d'apparition.
3. Écrivez les messages dans l'ordre, en nommant clairement chaque appel.
4. Ajoutez les retours et les cas d'erreur avec des fragments « alt ».
5. Vérifiez la cohérence avec le diagramme de classes (les opérations existent-elles ?).

Exemple complet : création d'un ticket

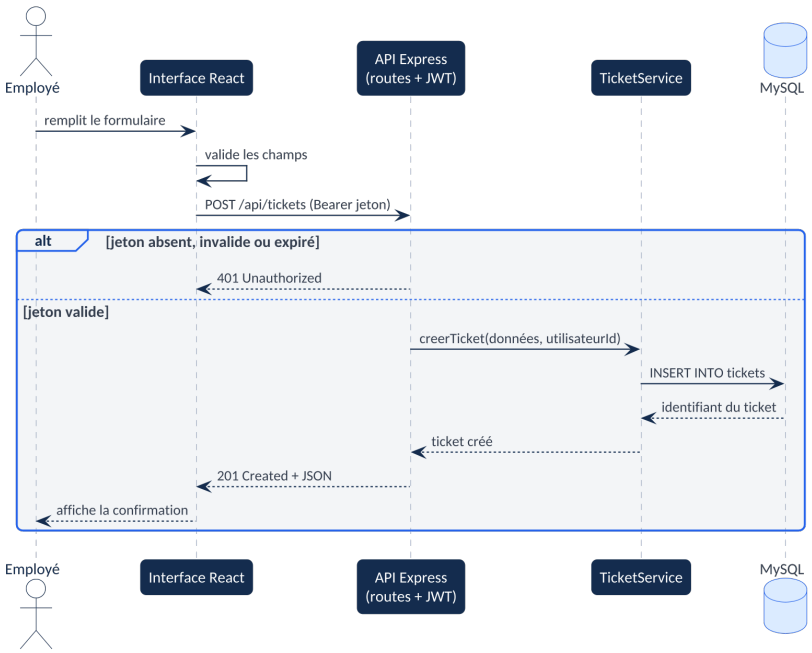


Figure 5 — Séquence « Créer un ticket » (SupportIT).

PLANTUML — SÉQUENCE

```
@startuml
actor "Employé" as U
participant "Interface React" as UI
participant "API Express\n(routes + JWT)" as API
participant "TicketService" as S
database "MySQL" as DB
U -> UI : remplit le formulaire
UI -> UI : valide les champs
UI -> API : POST /api/tickets (Bearer jeton)
alt jeton absent, invalide ou expiré
  API --> UI : 401 Unauthorized
else jeton valide
  API -> S : creerTicket(données, utilisateurId)
  S -> DB : INSERT INTO tickets
  DB --> S : identifiant du ticket
  S --> API : ticket créé
  API --> UI : 201 Created + JSON
  UI --> U : affiche la confirmation
end
@enduml
```

Erreurs fréquentes

- Un seul diagramme pour tout le système : un diagramme = un scénario.
- Oublier les cas d'erreur (401, 400).
- Messages vagues (« traite »), sans nom d'opération.
- Participants trop nombreux : regroupez les détails techniques.

Le diagramme ci-dessus est un exemple de séquence « conception » ; la séquence d'authentification est présentée au chapitre 5.

4.6 Diagramme d'activité

Aa

DÉFINITION

Diagramme d'activité

Représentation d'un **flux de travail** : actions, décisions, boucles et éventuellement parallélisme. Il ressemble à un organigramme (logigramme) en notation UML.

Utilité

Il décrit un processus métier ou un algorithme, et permet de vérifier avec le client que les règles de gestion sont comprises.

Composants

Élément	Rôle
Nœud initial / final	Début et fin du flux.
Action	Étape du processus (rectangle arrondi).
Décision	Losange avec conditions (gardes) sur les flèches.
Fusion	Réunion de chemins alternatifs.
Fork / join	Début et fin de flux parallèles.
Couloirs (swimlanes)	Attribuent les actions à un acteur.

Comment le construire

1. Choisissez le **processus** et ses bornes (début, fin).
2. Listez les actions dans l'ordre.
3. Repérez les décisions et nommez les gardes.
4. Ajoutez boucles et exceptions.
5. Vérifiez que chaque chemin mène à une fin.

Exemple complet : cycle de traitement d'un ticket

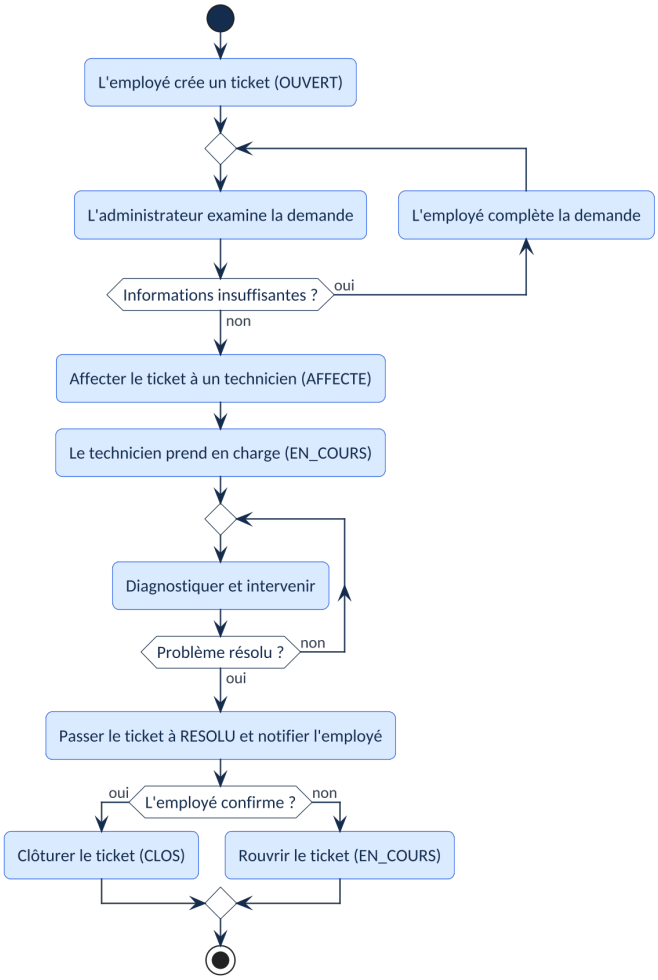


Figure 6 — Activité « Traiter un ticket » (SupportIT).

PLANTUML — ACTIVITÉ

```
@startuml
start
:L'employé crée un ticket (OUVERT);
repeat
  :L'administrateur examine la demande;
backward:L'employé complète la demande;
repeat while (Informations insuffisantes ?) is (oui) not (non)
:Affecter le ticket à un technicien (AFFECTE);
:Le technicien prend en charge (EN_COURS);
repeat
  :Diagnostiquer et intervenir;
repeat while (Problème résolu ?) is (non) not (oui)
:Passer le ticket à RESOLU et notifier l'employé;
if (L'employé confirme ?) then (oui)
  :Clôturer le ticket (CLOS);
else (non)
  :Rouvrir le ticket (EN_COURS);
endif
stop
@enduml
```

Erreurs fréquentes

- Décision sans toutes les sorties (un seul « oui »).
- Boucles infinies sans condition de sortie.
- Confondre activité (flux) et séquence (messages entre participants).
- Actions trop détaillées (une ligne de code) ou trop vagues.

4.7 Diagramme d'états-transitions

Aa **DÉFINITION**

Diagramme d'états-transitions

Description des **états** successifs d'un objet et des **événements** qui provoquent le passage d'un état à un autre. On l'utilise lorsqu'un objet a un cycle de vie important, ce qui est le cas du ticket.

Élément	Rôle
État	Situation stable de l'objet (OUVERT, EN_COURS…).
Transition	Passage d'un état à un autre, déclenché par un événement (et éventuellement une condition).
État initial / final	Début et fin de vie de l'objet.

Comment le construire : 1) listez les états possibles ; 2) listez les événements ; 3) reliez les états par des transitions autorisées ; 4) interdisez explicitement le reste (un ticket CLOS ne revient pas à OUVERT).

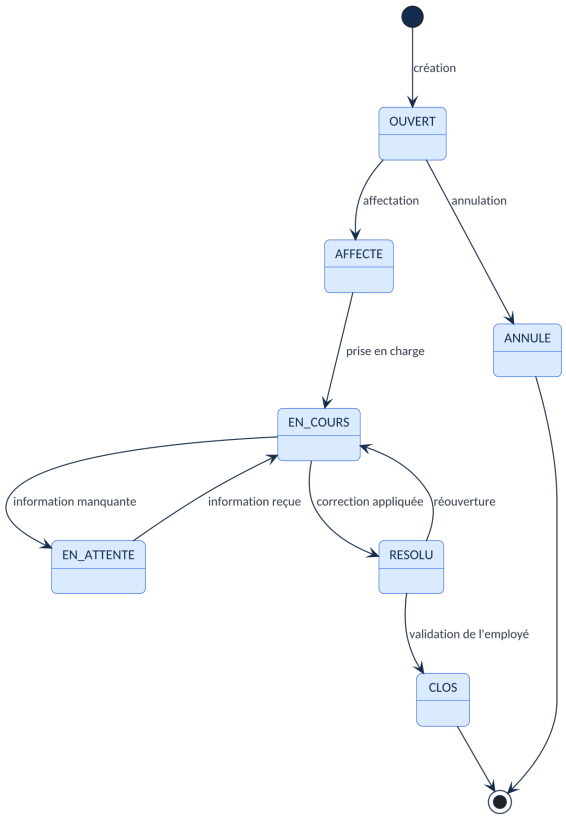


Figure 7 — États d'un ticket SupportIT.

PLANTUML — ÉTATS-TRANSITIONS

```
@startuml

[*] --> OUVERT : création
OUVERT --> AFFECTE : affectation
OUVERT --> ANNULE : annulation
AFFECTE --> EN_COURS : prise en charge
EN_COURS --> EN_ATTENTE : information manquante
EN_ATTENTE --> EN_COURS : information reçue
EN_COURS --> RESOLU : correction appliquée
RESOLU --> CLOS : validation de l'employé
RESOLU --> EN_COURS : réouverture
CLOS --> [*]
ANNULE --> [*]
@enduml
```

- **Erreurs fréquentes :** états qui sont en réalité des actions, transitions sans événement, états inaccessibles ou « puits » sans sortie.
- **Intérêt pour les tests :** chaque transition autorisée et chaque transition interdite devient un cas de test (chapitre 7).

4.8 Cohérence entre les diagrammes

Les diagrammes ne sont pas des dessins indépendants : ils décrivent le **même système**. Avant la validation V3, vérifiez :

CONTRÔLE DE COHÉRENCE UML	
ACTEURS ET CAS	
☐	Chaque acteur du cas d'utilisation apparaît dans le cahier des charges
☐	Chaque cas d'utilisation couvre au moins une exigence
CLASSES ET SCÉNARIOS	
☐	Les participants des séquences correspondent à des classes ou composants
☐	Les opérations appelées existent dans le diagramme de classes
☐	Les statuts de l'état-transition sont les mêmes que l'énumération Statut
VOCABULAIRE	
☐	Un même mot désigne un même concept partout (ticket, intervention…)
☐	Les identifiants (UC-03, EX-03) sont cités dans les documents

À RETENIR

Ce que vous retenez

Chaque diagramme répond à une question : qui fait quoi, quelles données, quel enchaînement, quel cycle de vie.

Construisez-les à partir du cahier des charges, avec un seul projet fil rouge cohérent.

PlantUML permet de versionner et de reproduire vos diagrammes.

M.D.S

CHAPITRE 05

Phase 4 : conception technique et architecture

05

Passer de la conception fonctionnelle à l'architecture technique : styles d'architecture, stack, base de données, sécurité, rôles et organisation du projet.

AU PROGRAMME

5.1 De la conception fonctionnelle à la conception technique

5.2 Les styles d'architecture

5.3 Choisir sa stack technologique

5.4 Architecture retenue pour SupportIT

5.5 Modélisation de la base de données

5.6 Conception de l'API REST

5.7 Sécurité, authentification et rôles

5.8 Organisation des dossiers du projet

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : dossier de conception technique (validation V4) · Toolbox : modèle n°4

5.1 De la conception fonctionnelle à la conception technique

La conception fonctionnelle décrit **ce que fait** le système ; la conception technique décrit **comment il est construit** : architecture, technologies, base de données, sécurité, organisation du code. Chaque choix doit être **justifié** par une exigence ou une contrainte.

Entrée (phase précédente)	Décision technique	Livrable
Cas d'utilisation, exigences	Découpage en modules et en routes API	Architecture logique
Diagramme de classes	Schéma de base de données	Modèle entité-association et relationnel
Exigences non fonctionnelles	Technologies, sécurité, déploiement	Choix de la stack justifié
Contraintes de l'entreprise	Environnement, hébergement	Plan d'architecture et de déploiement

5.2 Les styles d'architecture

Style	Principe	Avantages	Limites / quand l'éviter
Monolithique	Une seule application déployable contenant toutes les fonctions.	Simple à développer, tester, déployer.	Grossit avec le temps ; peu adapté à de très grandes équipes.
En couches	Séparation en couches (présentation, métier, accès aux données) qui ne communiquent que de haut en bas.	Responsabilités claires ; testabilité.	Risque de couches « traversantes » inutiles.
MVC	Modèle (données), Vue (affichage), Contrôleur (coordination).	Structure connue, adaptée aux applications web.	Le contrôleur peut devenir trop gros.
Client-serveur	Un client envoie des requêtes à un serveur qui répond.	Centralise la logique et les données.	Dépend du réseau et du serveur.
REST	API qui expose des ressources via les verbes HTTP, en JSON.	Standard, simple, indépendant du client.	Pas adapté à tout (temps réel intensif).

À SAVOIR

Ces styles se combinent

SupportIT est un **monolithe en couches** (côté serveur) qui suit le modèle **client-serveur** et expose une **API REST**. Le front-end React joue le rôle de « vue » ; côté serveur, les contrôleurs, services et dépôts reprennent l'esprit du MVC. Pour un PFE, un monolithe bien structuré est généralement le meilleur choix : il évite la complexité inutile des microservices.

Couche (serveur)	Responsabilité	Exemple
Routes	Associer une URL et un verbe à un contrôleur.	POST /api/tickets

Couche (serveur)	Responsabilité	Exemple
Middlewares	Traitements transversaux : authentification, rôles, validation.	verifierJeton, autoriser('ADMIN')
Contrôleurs	Lire la requête, appeler le service, renvoyer la réponse HTTP.	ticketController.creer
Services	Règles métier.	creerTicket, changerStatut
Repositories	Accès à la base (requêtes SQL).	ticketRepository.insert

5.3 Choisir sa stack technologique

Une stack doit être **cohérente** : on n'empile pas toutes les technologies dans un même projet. Voici trois combinaisons classiques ; chacune est un ensemble complet.

	Stack A (fil rouge)	Stack B	Stack C
Front-end	React.js	Angular	React.js
Back-end	Node.js + Express.js	Java + Spring Boot	Node.js + Express.js
Base de données	MySQL	PostgreSQL	MongoDB
Authentification	JWT	JWT (Spring Security)	JWT
Points forts	Un seul langage (JavaScript) ; très répandue ; démarrage rapide.	Structure rigoureuse, typage fort, écosystème d'entreprise.	Schéma flexible, rapide à prototyper.
Points d'attention	Rigueur de structure à imposer soi-même.	Courbe d'apprentissage plus longue.	Cohérence des données à gérer dans l'application.
Quand la choisir	Application web de gestion avec données relationnelles.	Contexte Java en entreprise, besoin de robustesse.	Données peu structurées ou évolutives.



CONSEIL PROFESSIONNEL

Justifiez votre choix avec des critères

Besoin fonctionnel, contraintes de l'entreprise, compétences, durée, écosystème, sécurité. Pour SupportIT : données fortement relationnelles (utilisateurs, tickets, commentaires) → base relationnelle ; équipe d'un étudiant maîtrisant JavaScript → Stack A.



ATTENTION

Ne mélangez pas

Évitez des assemblages incohérents comme « Angular + React + Express + Spring Boot + MySQL + MongoDB » dans un même projet : chaque technologie ajoutée augmente le travail, les risques et le volume à documenter.

5.4 Architecture retenue pour SupportIT

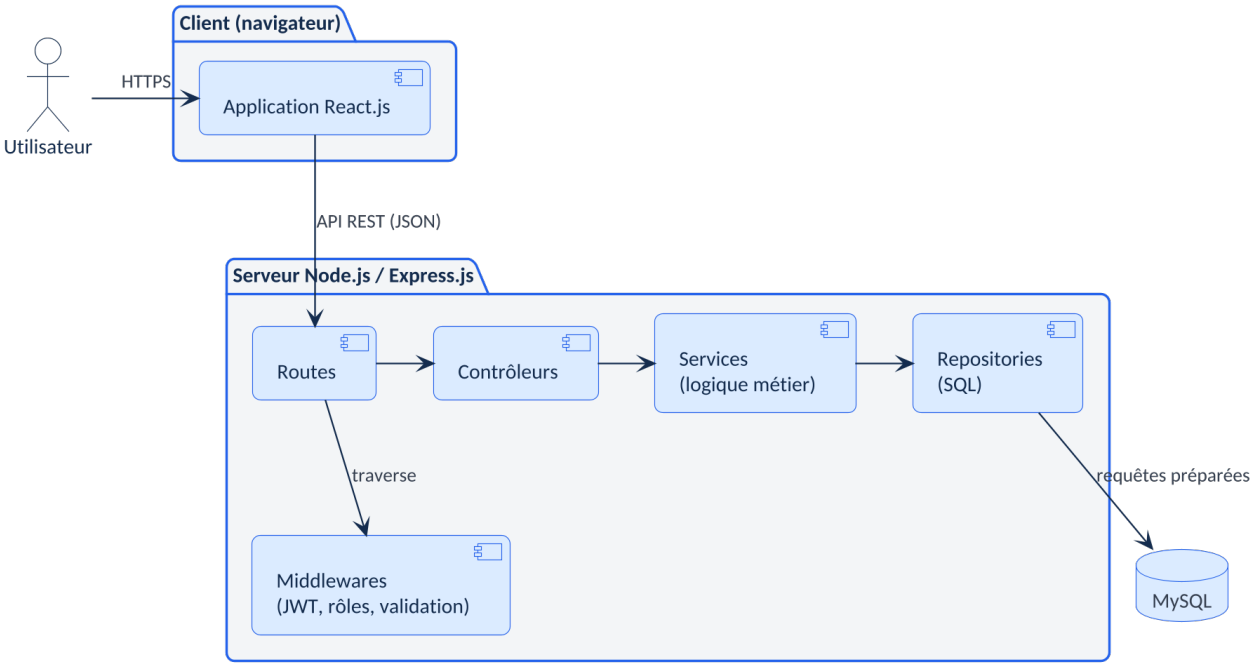


Figure 8 — Architecture logique de SupportIT (Stack A).

PLANTUML — ARCHITECTURE

```
@startuml
actor Utilisateur as U
package "Client (navigateur)" {
  [Application React.js] as FE
}
package "Serveur Node.js / Express.js" {
  [Routes] as R
  [Contrôleurs] as C
  [Services\n(logique métier)] as S
  [Repositories\n(SQL)] as RP
  [Middlewares\n(JWT, rôles, validation)] as MWS
}
database "MySQL" as DB
U -right-> FE : HTTPS
FE -down-> R : API REST (JSON)
R -right-> C
C -right-> S
S -right-> RP
R -down-> MWS : traverse
RP -down-> DB : requêtes préparées
@enduml
```

Le navigateur charge l'application React ; celle-ci dialogue avec l'API Express en JSON. Chaque requête traverse les middlewares (jeton, rôle, validation) avant d'atteindre le contrôleur, puis le service et le repository qui interroge MySQL par requêtes préparées.

5.5 Modélisation de la base de données

Du diagramme de classes au modèle entité-association

Aa

DÉFINITION

Modèle entité-association (ER / MCD)

Représentation des **entités** (objets métier), de leurs **attributs** et des **associations** avec leurs cardinalités. Il se traduit ensuite en **modèle relationnel** : tables, clés primaires et clés étrangères.

Règle de passage	Application
Une classe/entité devient une table	Ticket → tickets
Une association 1—N donne une clé étrangère côté « N »	Ticket (N) porte createur_id
Une énumération devient un champ ENUM ou une table de référence	statut ENUM(...)
Une association N—N donne une table de liaison	(non utilisée dans SupportIT)

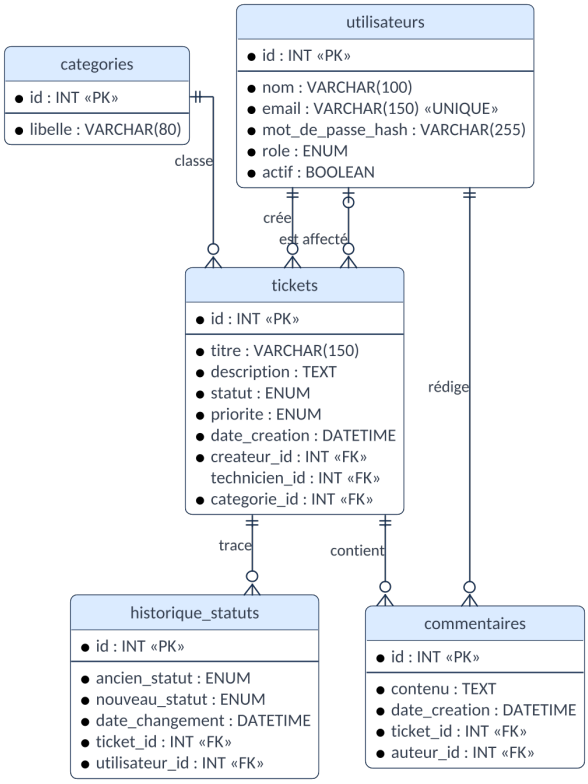


Figure 9 — Diagramme entité-association de SupportIT (notation « pattes d'oie »).

PLANTUML — ENTITÉ-ASSOCIATION

```
@startuml
hide circle
skinparam linetype ortho
entity "utilisateurs" as u {
  * id : INT <<PK>>
  --
  * nom : VARCHAR(100)
  * email : VARCHAR(150) <<UNIQUE>>
  * mot_de_passe_hash : VARCHAR(255)
  * role : ENUM
  * actif : BOOLEAN
}
entity "categories" as c {
  * id : INT <<PK>>
  --
  * libelle : VARCHAR(80)
}
entity "tickets" as t {
  * id : INT <<PK>>
  --
  * titre : VARCHAR(150)
  * description : TEXT
  * statut : ENUM
  * priorite : ENUM
  * date_creation : DATETIME
  * createur_id : INT <<FK>>
  technicien_id : INT <<FK>>
  * categorie_id : INT <<FK>>
}
entity "commentaires" as m {
  * id : INT <<PK>>
  --
  * contenu : TEXT
  * date_creation : DATETIME
  * ticket_id : INT <<FK>>
  * auteur_id : INT <<FK>>
}
entity "historique_statuts" as h {
  * id : INT <<PK>>
  --
  * ancien_statut : ENUM
  * nouveau_statut : ENUM
  * date_changement : DATETIME
  * ticket_id : INT <<FK>>
  * utilisateur_id : INT <<FK>>
}
u ||--o{ t : crée
u |o--o{ t : est affecté
c ||--o{ t : classe
t ||--o{ m : contient
u ||--o{ m : rédige
t ||--o{ h : trace
@enduml
```

Diagramme relationnel

Table	Colonnes (PK, FK)	Contraintes
utilisateurs	**id**, nom, email, mot_de_passe_hash, role, actif	email UNIQUE
categories	**id**, libelle	libelle UNIQUE
tickets	**id**, titre, description, statut, priorite, date_creation, *createur_id*, *technicien_id*, *categorie_id*	FK vers utilisateurs (2) et categories ; technicien_id peut être NULL
commentaires	**id**, contenu, date_creation, *ticket_id*, *auteur_id*	FK ; suppression en cascade avec le ticket
historique_statuts	**id**, ancien_statut, nouveau_statut, date_changement, *ticket_id*, *utilisateur_id*	FK ; table de traçabilité

Gras : clé primaire ; italique : clé étrangère. Selon votre établissement, la modélisation peut être demandée en Merise (MCD, MLD, MPD) ou en UML : suivez la méthode exigée, les principes restent les mêmes.

MYSQL — EXTRAIT DU SCHÉMA

```
CREATE TABLE utilisateurs (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nom VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  mot_de_passe_hash VARCHAR(255) NOT NULL,
  role ENUM('ADMIN','TECHNICIEN','EMPLOYE') NOT NULL,
  actif BOOLEAN NOT NULL DEFAULT TRUE
);
CREATE TABLE tickets (
  id INT PRIMARY KEY AUTO_INCREMENT,
  titre VARCHAR(150) NOT NULL,
  description TEXT NOT NULL,
  statut ENUM('OUVERT','AFFECTE','EN_COURS','EN_ATTENTE','RESOLU','CLOS','ANNULE')
         NOT NULL DEFAULT 'OUVERT',
  priorite ENUM('BASSE','NORMALE','HAUTE','CRITIQUE') NOT NULL DEFAULT 'NORMALE',
  date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  createur_id INT NOT NULL,
  technicien_id INT NULL,
  categorie_id INT NOT NULL,
  FOREIGN KEY (createur_id) REFERENCES utilisateurs(id),
  FOREIGN KEY (technicien_id) REFERENCES utilisateurs(id),
  FOREIGN KEY (categorie_id) REFERENCES categories(id)
);
```

Et avec PostgreSQL ou MongoDB ?

Avec **PostgreSQL** (Stack B), le modèle relationnel reste le même ; seules la syntaxe des types (par exemple SERIAL ou identité, TIMESTAMP) et des énumérations changent. Avec **MongoDB** (Stack C), on privilégie des documents : un ticket peut contenir ses commentaires *imbriqués* lorsqu'ils sont toujours lus avec lui, et l'historique des statuts peut être un tableau dans le même document. Les relations avec les utilisateurs se font par référence (identifiant).

MONGODB — DOCUMENT TICKET (EXEMPLE ILLUSTRATIF)

```
{ "titre": "Écran noir au démarrage", "statut": "OUVERT", "priorite": "HAUTE",
  "createurId": "u_102", "technicienId": null,
  "commentaires": [
    { "auteurId": "u_102", "contenu": "Problème persistant", "date": "2026-03-04T09:12:00Z" }
  ],
  "historiqueStatuts": [
    { "ancien": null, "nouveau": "OUVERT", "date": "2026-03-04T09:10:00Z" }
  ] }
```

5.6 Conception de l'API REST

Aa

DÉFINITION

API REST

Interface qui expose des ressources identifiées par des URL et manipulées avec les verbes HTTP (GET, POST, PUT, PATCH, DELETE), généralement en JSON. REST est un style d'architecture décrit par Roy Fielding (2000).

Méthode et route	Rôle	Accès	Exigence
POST /api/auth/login	Authentifier	Public	EX-01

Méthode et route	Rôle	Accès	Exigence
GET /api/utilisateurs	Lister les comptes	ADMIN	EX-02
POST /api/utilisateurs	Créer un compte	ADMIN	EX-02
POST /api/tickets	Créer un ticket	EMPLOYE	EX-03
GET /api/tickets	Lister (filtré selon le rôle)	Tous	EX-04, EX-11
PATCH /api/tickets/:id/affectation	Affecter à un technicien	ADMIN	EX-05
PATCH /api/tickets/:id/statut	Changer le statut	TECHNICIEN affecté	EX-06
POST /api/tickets/:id/commentaires	Commenter	Participants	EX-07
GET /api/tickets/:id/historique	Historique des statuts	Participants	EX-08
GET /api/statistiques	Tableau de bord	ADMIN	EX-09, EX-10

- Noms de ressources au pluriel, sans verbe ; **codes HTTP cohérents** (200, 201, 400, 401, 403, 404, 409).
- Format d'erreur homogène : `{ "message": "…", "code": "…" }` sans détail technique sensible.
- Documenter l'API (chapitre 9) et l'alimenter à partir de cette table.

5.7 Sécurité, authentification et rôles

Aa

DÉFINITION

Authentification et autorisation

Authentifier : vérifier l'identité d'un utilisateur. **Autoriser** : décider de ce qu'il a le droit de faire. **JWT** (JSON Web Token, RFC 7519) est un jeton signé que le serveur délivre après connexion et que le client renvoie à chaque requête.

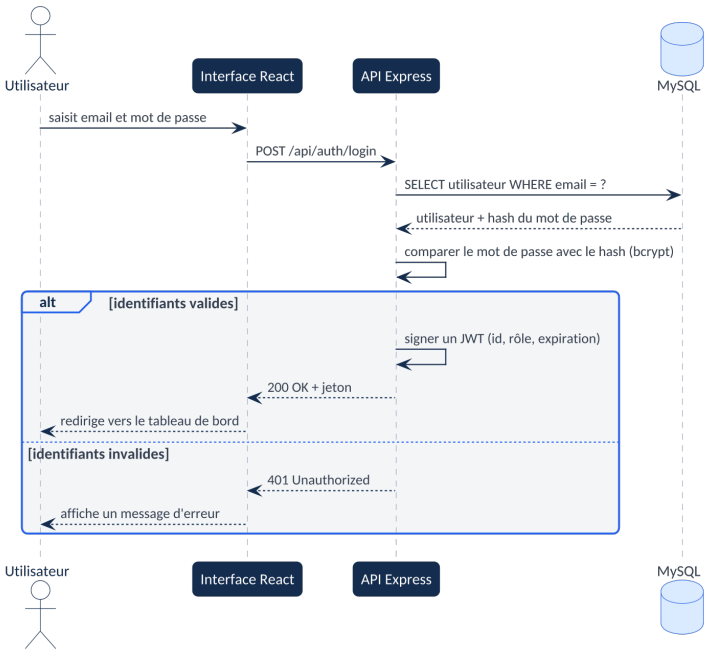


Figure 10 — Authentification par JWT dans SupportIT.

Fonction / rôle	ADMIN	TECHNICIEN	EMPLOYE
Se connecter	Oui	Oui	Oui
Gérer les utilisateurs	Oui	Non	Non
Créer un ticket	Non (optionnel)	Non	Oui
Voir ses propres tickets	—	Affectés à lui	Les siens
Voir tous les tickets	Oui	Non	Non
Affecter un ticket	Oui	Non	Non
Changer le statut	Oui	Ses tickets	Non (sauf valider la résolution)
Commenter	Oui	Ses tickets	Ses tickets
Tableau de bord	Oui	Non	Non

Cette **matrice des rôles et permissions** est un livrable de conception : elle sera implémentée par un middleware et vérifiée par des tests (chapitre 7).

- **Mots de passe :** hachés avec bcrypt (ou Argon2) ; jamais en clair ni chiffrés de façon réversible.
- **Jeton :** durée de vie limitée ; clé de signature secrète stockée en variable d'environnement.
- **Validation :** toute entrée est validée côté serveur ; requêtes SQL préparées contre les injections.
- **Autorisation :** vérifier le rôle *et* la propriété de la ressource (un employé ne lit que ses tickets).
- **Transport :** HTTPS en production ; CORS restreint aux origines nécessaires.



ATTENTION

Les contrôles côté interface ne protègent rien

Masquer un bouton dans React ne sécurise pas l'action : toutes les règles d'accès doivent être appliquées par l'API.

5.8 Organisation des dossiers du projet

Décidez de l'organisation **avant** de coder : elle reflète l'architecture. Voici la structure retenue pour SupportIT (détail au chapitre 6).

ARBORESCENCE PRÉVUE

```
supportit/
├── backend/
│   ├── src/
│   │   ├── config/        (connexion BDD, variables d'environnement)
│   │   ├── routes/        (auth, utilisateurs, tickets, statistiques)
│   │   ├── controllers/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── middlewares/   (verifierJeton, autoriser, validation, erreurs)
│   │   └── app.js
│   ├── tests/
│   ├── .env.example
│   └── package.json
├── frontend/
│   ├── src/ (components, pages, services, hooks, App.jsx)
│   └── package.json
├── docs/                  (analyse, cahier des charges, UML, API, rapport)
├── database/              (schema.sql, seed.sql)
└── README.md
```

CONTRÔLE DU DOSSIER DE CONCEPTION TECHNIQUE (V4)

CONTENU	
☐	Architecture décrite et justifiée
☐	Stack choisie avec critères
☐	Modèle ER et schéma relationnel cohérents avec le diagramme de classes
☐	Routes API listées et reliées aux exigences
☐	Matrice rôles / permissions
☐	Organisation du code et conventions
VALIDATION	
☐	Relecture par l'encadrant
☐	Compte rendu ou signature

À RETENIR

Ce que vous retenez

Chaque choix technique est justifié par une exigence ou une contrainte.

Une architecture cohérente vaut mieux qu'un empilement de technologies.

Base de données, API, sécurité et rôles se conçoivent ensemble, avant le développement.

M.D.S

CHAPITRE 06

Phase 5 : préparation de l'environnement et développement

06

Installer les outils, initialiser le projet, organiser et écrire le code (API REST, JWT, validation, rôles, interface React) et travailler proprement avec Git et GitHub.

AU PROGRAMME

6.1 Installer et préparer les outils

6.2 Configurer la base de données

6.3 Initialiser le projet

6.4 Organisation du code, conventions et variables d'environnement

6.5 Développement back-end : API, base, erreurs

6.6 Authentification JWT, validation et rôles

6.7 Développement front-end avec React

6.8 Bonnes pratiques Git

6.9 Un dépôt GitHub professionnel

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : code source versionné et fonctionnel · Toolbox : modèle n°11 (README)

6.1 Installer et préparer les outils

La phase de développement ne commence qu'une fois les phases précédentes validées. Commencez par un environnement propre et reproductible.

Outil	Rôle	Vérification
Node.js (version LTS) et npm	Exécuter le back-end et gérer les paquets.	node -v · npm -v
Git	Gérer les versions du code.	git --version
MySQL Server (et un client : ligne de commande ou interface)	Base de données.	mysql --version
VS Code (ou IntelliJ IDEA pour la Stack B Java)	Éditeur de code.	—
Postman ou outil équivalent	Tester l'API manuellement.	—
Navigateur récent avec outils de développement	Déboguer le front-end.	—

Préparer VS Code

- Extensions utiles : ESLint (qualité du code), Prettier (formatage automatique), un client PlantUML pour les diagrammes, un client MySQL ou une extension de base de données.
- Activer « Formater à l'enregistrement » pour garder un style homogène.
- Pour IntelliJ IDEA (Stack B) : importer le projet Maven ou Gradle et vérifier la version du JDK.



CONSEIL PROFESSIONNEL

Installez les versions supportées

Choisissez des versions stables et notez-les dans le README (« Node.js 20 ou supérieur », etc.). Cela évite le fameux « ça marche chez moi ».

6.2 Configurer la base de données

Créez la base, un utilisateur dédié à l'application avec des **droits limités** (principe du moindre privilège), puis chargez le schéma conçu au chapitre 5.

MYSQL — BASE ET UTILISATEUR APPLICATIF

```
CREATE DATABASE supportit CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'supportit_app'@'localhost' IDENTIFIED BY 'remplacer_par_un_mot_de_passe_fort';
GRANT SELECT, INSERT, UPDATE, DELETE ON supportit.* TO 'supportit_app'@'localhost';
FLUSH PRIVILEGES;
```

Chargez ensuite `database/schema.sql` (tables) puis `database/seed.sql` (données de test : catégories, un compte par rôle). Le schéma de création des tables doit être exécuté avec un compte disposant des droits de création, distinct du compte applicatif.

ATTENTION

Données de démonstration

Les comptes du fichier seed.sql servent aux tests : ils doivent être supprimés ou leurs mots de passe changés avant toute mise en ligne.

6.3 Initialiser le projet

TERMINAL — INITIALISATION DE SUPPORTIT

```
mkdir supportit && cd supportit
git init
mkdir backend docs database
cd backend
npm init -y
npm install express mysql2 bcryptjs jsonwebtoken dotenv cors helmet express-validator
npm install --save-dev nodemon jest supertest
cd ..
npm create vite@latest frontend -- --template react
cd frontend && npm install && npm install axios react-router-dom
```

Dans `backend/package.json`, ajoutez des scripts : `"dev": "nodemon src/server.js"`, `"start": "node src/server.js"`, `"test": "jest"`. Faites ensuite un premier commit « chore: initialiser le projet » avec un fichier `.gitignore`.

.GITIGNORE

```
node_modules/
.env
dist/
build/
coverage/
*.log
.DS_Store
```

6.4 Organisation du code, conventions et variables d'environnement

Séparation des responsabilités

Couche	Doit	Ne doit pas
Route	Déclarer URL, middlewares et contrôleur.	Contenir de la logique métier.
Contrôleur	Lire la requête, appeler le service, répondre.	Écrire du SQL.
Service	Appliquer les règles métier (transitions de statut…).	Connaître HTTP (req, res).
Repository	Exécuter les requêtes SQL.	Appliquer des règles métier.

Conventions de nommage

Élément	Convention	Exemple
Fichiers JavaScript	camelCase	ticketService.js
Composants React	PascalCase	ListeTickets.jsx
Variables et fonctions	camelCase, verbes pour les fonctions	creerTicket()
Constantes	MAJUSCULES_AVEC_TIRETS_BAS	TRANSITIONS
Tables et colonnes SQL	snake_case, tables au pluriel	historique_statuts
Routes API	minuscules, ressources au pluriel	/api/tickets/:id/statut
Branches Git	type/description	feature/creation-ticket

Variables d'environnement

Les paramètres qui changent selon l'environnement, et surtout les **secrets**, ne se mettent jamais dans le code. On les place dans un fichier `.env` ignoré par Git, et on versionne un modèle `.env.example` sans valeurs sensibles.

BACKEND/.ENV.EXAMPLE

```
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=supportit_app
DB_PASSWORD=
DB_NAME=supportit
JWT_SECRET=
JWT_EXPIRES_IN=2h
CORS_ORIGIN=http://localhost:5173
```

6.5 Développement back-end : API, base, erreurs

Voici le squelette de l'application, suivi d'une fonctionnalité complète (création de ticket) du route au repository. Les extraits sont **pédagogiques** : ils illustrent la structure et ne remplacent pas un code complet testé.

NODE.JS / EXPRESS — CONFIGURATION ET APPLICATION

```
// src/config/db.js
const mysql = require('mysql2/promise');
module.exports = mysql.createPool({
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT) || 3306,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
});

// src/app.js
require('dotenv').config();
const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const routes = require('./routes');
const { gestionErreurs } = require('./middlewares/erreurs');

const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.CORS_ORIGIN }));
app.use(express.json());
app.use('/api', routes);
app.use(gestionErreurs);
module.exports = app;

// src/server.js
const app = require('./app');
app.listen(process.env.PORT || 3000);
```

Gestion centralisée des erreurs

MIDDLEWARE D'ERREURS

```
// src/middlewares/erreurs.js
class ErreurApp extends Error {
  constructor(statut, message) { super(message); this.statut = statut; }
}
function gestionErreurs(err, req, res, next) {
  const statut = err.statut || 500;
  if (statut === 500) console.error(err);   // détail dans les journaux uniquement
  res.status(statut).json({ message: statut === 500 ? 'Erreur interne du serveur' : err.message });
}
module.exports = { ErreurApp, gestionErreurs };
```

Une fonctionnalité complète : créer un ticket

FONCTIONNALITÉ « CRÉER UN TICKET » (EXTRAITS)

```
// src/routes/tickets.js
router.post('/', verifierJeton, autoriser('EMPLOYE'), regles.creerTicket, valider, ctrl.creer);

// src/controllers/ticketController.js
exports.creer = async (req, res, next) => {
  try {
    const ticket = await ticketService.creerTicket(req.body, req.utilisateur.id);
    res.status(201).json(ticket);
  } catch (e) { next(e); }
};

// src/services/ticketService.js
const TRANSITIONS = {
  OUVERT: ['AFFECTE', 'ANNULE'], AFFECTE: ['EN_COURS'], EN_COURS: ['EN_ATTENTE', 'RESOLU'],
  EN_ATTENTE: ['EN_COURS'], RESOLU: ['CLOS', 'EN_COURS'], CLOS: [], ANNULE: [],
};
const transitionAutorisee = (de, vers) => (TRANSITIONS[de] || []).includes(vers);

async function creerTicket(donnees, utilisateurId) {
  const id = await ticketRepository.inserer({
    ...donnees, priorite: donnees.priorite || 'NORMALE', createurId: utilisateurId });
  await ticketRepository.ajouterHistorique(id, null, 'OUVERT', utilisateurId);
  return ticketRepository.trouverParId(id);
}

// src/repositories/ticketRepository.js
async function inserer({ titre, description, priorite, categorieId, createurId }) {
  const [res] = await pool.query(
    'INSERT INTO tickets (titre, description, priorite, categorie_id, createur_id) ' +
    'VALUES (?, ?, ?, ?, ?)',
    [titre, description, priorite, categorieId, createurId]);
  return res.insertId;
}
```

À SAVOIR

Le lien avec la conception

La constante TRANSITIONS reprend exactement le diagramme d'états du chapitre 4, et la route correspond à la ligne « POST /api/tickets » du tableau de l'API. Si le code diverge de la conception, mettez à jour le document (et revalidez) plutôt que de laisser les deux se contredire.

6.6 Authentification JWT, validation et rôles

AUTHENTIFICATION JWT ET GESTION DES RÔLES

```
// src/services/authService.js  — connexion
async function connecter(email, motDePasse) {
  const [lignes] = await pool.query(
    'SELECT id, nom, mot_de_passe_hash, role, actif FROM utilisateurs WHERE email = ?', [email]);
  const u = lignes[0];
  const ok = u && u.actif && await bcrypt.compare(motDePasse, u.mot_de_passe_hash);
  if (!ok) throw new ErreurApp(401, 'Identifiants invalides');  // message générique
  const jeton = jwt.sign({ id: u.id, role: u.role }, process.env.JWT_SECRET,
                         { expiresIn: process.env.JWT_EXPIRES_IN || '2h' });
  return { jeton, utilisateur: { id: u.id, nom: u.nom, role: u.role } };
}

// src/middlewares/auth.js  — jeton et rôles
function verifierJeton(req, res, next) {
  const [type, jeton] = (req.headers.authorization || '').split(' ');
  if (type !== 'Bearer' || !jeton) return next(new ErreurApp(401, 'Jeton manquant'));
  try { req.utilisateur = jwt.verify(jeton, process.env.JWT_SECRET); next(); }
  catch (e) { next(new ErreurApp(401, 'Jeton invalide ou expiré')); }
}
const autoriser = (...roles) => (req, res, next) =>
  roles.includes(req.utilisateur.role) ? next() : next(new ErreurApp(403, 'Accès interdit'));
```

VALIDATION DES DONNÉES

```
// src/middlewares/validation.js
const { body, validationResult } = require('express-validator');
const regles = {
  creerTicket: [
    body('titre').trim().isLength({ min: 5, max: 150 }).withMessage('Titre de 5 à 150 caractères'),
    body('description').trim().notEmpty().withMessage('Description obligatoire'),
    body('categorieId').isInt({ min: 1 }).withMessage('Catégorie invalide'),
    body('priorite').optional().isIn(['BASSE', 'NORMALE', 'HAUTE', 'CRITIQUE']),
  ],
};
function valider(req, res, next) {
  const erreurs = validationResult(req);
  return erreurs.isEmpty() ? next() : next(new ErreurApp(400, erreurs.array()[0].msg));
}
```

- À la création d'un compte, le mot de passe est haché avec `bcrypt.hash(motDePasse, 10)` avant insertion.
- La route `PATCH /api/tickets/:id/statut` doit aussi vérifier que le technicien est **celui à qui le ticket est affecté**, et que `transitionAutorisee` renvoie vrai.

6.7 Développement front-end avec React

Le front-end se développe à partir des maquettes et des cas d'utilisation. Organisez-le en pages (écrans), composants réutilisables et services (appels API).

REACT — SERVICE API ET PAGE DE CONNEXION (EXTRAITS)

```
// src/services/api.js
import axios from 'axios';
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });
api.interceptors.request.use((config) => {
  const jeton = localStorage.getItem('jeton');
  if (jeton) config.headers.Authorization = `Bearer ${jeton}`;
  return config;
});
export default api;

// src/pages/Login.jsx
export default function Login() {
  const [email, setEmail] = useState('');
  const [motDePasse, setMotDePasse] = useState('');
  const [erreur, setErreur] = useState('');
  const navigate = useNavigate();

  async function soumettre(e) {
    e.preventDefault();
    try {
      const { data } = await api.post('/auth/login', { email, motDePasse });
      localStorage.setItem('jeton', data.jeton);
      navigate('/tableau-de-bord');
    } catch (err) {
      setErreur(err.response?.data?.message || 'Connexion impossible');
    }
  }
  // ... formulaire : champs email, mot de passe, bouton, message d'erreur
}
```

À SAVOIR

Stockage du jeton

Stocker le jeton dans localStorage est simple et fréquent en PFE, mais il est exposé en cas de faille XSS. Une alternative plus robuste est un cookie httpOnly. Quel que soit votre choix, justifiez-le dans le dossier de conception.

- Protégez les pages par un composant « route privée » qui redirige vers la connexion et vérifie le rôle (confort d'usage : la vraie sécurité est côté API).
- Affichez des états de chargement et des messages d'erreur clairs.
- Pour la Stack B : Angular fournit services, guards et intercepteurs pour les mêmes besoins.

6.8 Bonnes pratiques Git

Commande	Rôle	Exemple
git init	Créer un dépôt local.	git init
git add	Préparer les fichiers à enregistrer.	git add src/routes/tickets.js
git commit	Enregistrer une version avec un message.	git commit -m "feat(tickets): créer un ticket"
git branch	Lister ou créer une branche.	git branch feature/creation-ticket
git switch	Changer de branche.	git switch feature/creation-ticket
git merge	Fusionner une branche.	git merge feature/creation-ticket

Commande	Rôle	Exemple
git push	Envoyer les commits vers GitHub.	git push origin feature/creation-ticket
git pull	Récupérer et intégrer les changements distants.	git pull origin develop

Un flux de travail simple

1. **Partir de develop à jour**
   git switch develop && git pull
2. **Créer une branche par fonctionnalité**
   git switch -c feature/affectation-ticket
3. **Commits petits et fréquents**
   Un commit = une idée, avec un message de type « feat(tickets): affecter un ticket ».
4. **Pousser et ouvrir une pull request**
   Relecture (par l'encadrant, un pair ou vous-même après un temps de recul), tests, puis fusion dans develop.
5. **Marquer les versions**
   Sur main : git tag v1.0.0 à chaque livraison validée.

À faire	À éviter
• Pousser chaque jour (sauvegarde). • Messages de commit explicites. • Résoudre les conflits en comprenant les deux versions. • Ne jamais committer de secrets.	• Un énorme commit « tout » à la fin. • Travailler directement sur main. • Forcer un push (--force) sur une branche partagée. • Supprimer l'historique pour « cacher » une erreur.

6.9 Un dépôt GitHub professionnel

- **README** soigné (présentation, installation, lancement, tests, captures d'écran) — modèle en boîte à outils.
- **Branches** : main (versions livrées), develop (intégration), feature/* (travail).
- **Issues** alignées sur les exigences (EX-03 → issue « Créer un ticket »), regroupées en **milestones** par phase.
- **Étiquettes (releases)** : v0.1.0 (conception validée), v0.5.0 (fonctions principales), v1.0.0 (livraison).
- **Dossier docs/** avec les livrables de chaque phase, versionnés avec le code.
- **Protection de main** : fusion uniquement par pull request si l'option est disponible.

ARBORESCENCE COMPLÈTE DE SUPPORTIT

```
supportit/
├── README.md
├── .gitignore
├── docs/
│   ├── 01-analyse-besoins.pdf       02-cahier-des-charges.pdf
│   ├── 03-conception-fonctionnelle/ (diagrammes .puml et images)
│   ├── 04-conception-technique.pdf  05-rapport-tests.pdf
│   └── api.md
├── database/
│   ├── schema.sql
│   └── seed.sql
├── backend/
│   ├── package.json
│   ├── .env.example
│   ├── src/
│   │   ├── app.js            server.js
│   │   ├── config/db.js
│   │   ├── routes/           index.js  auth.js  tickets.js  utilisateurs.js  statistiques.js
│   │   ├── controllers/      authController.js  ticketController.js ...
│   │   ├── services/         authService.js  ticketService.js ...
│   │   ├── repositories/     utilisateurRepository.js  ticketRepository.js ...
│   │   └── middlewares/      auth.js  validation.js  erreurs.js
│   └── tests/                auth.test.js  tickets.test.js  transitions.test.js
└── frontend/
    ├── package.json          vite.config.js
    ├── .env.example          (VITE_API_URL)
    └── src/
        ├── main.jsx          App.jsx
        ├── services/         api.js
        ├── context/          AuthContext.jsx
        ├── components/       Navbar.jsx  RoutePrivee.jsx  TicketCard.jsx
        └── pages/            Login.jsx  MesTickets.jsx  NouveauTicket.jsx
                              DetailTicket.jsx  Administration.jsx  TableauDeBord.jsx
```



À RETENIR

Ce que vous retenez

L'environnement est préparé et documenté avant d'écrire du code.

Le code reflète la conception : couches, routes, états, rôles.

Git quotidien, branches, messages clairs, aucun secret dans le dépôt.

M.D.S

CHAPITRE 07

Phase 6 : tests et validation

07

Prévoir les tests dès la conception, construire un plan de test, rédiger des cas de test, traiter les anomalies et valider les exigences.

AU PROGRAMME

7.1 Pourquoi prévoir les tests dès la conception

7.2 Les types de tests

7.3 Construire un plan de test

7.4 Scénarios et cas de test

7.5 Automatiser : exemples de tests

7.6 Identifier, corriger et suivre les bugs

7.7 Documenter les résultats et valider

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : rapport de tests et validation (V6) · Toolbox : modèle n°9

7.1 Pourquoi prévoir les tests dès la conception

Dans la cascade, les tests arrivent tard : une erreur découverte à ce stade coûte cher. La parade est de **préparer les tests pendant les phases précédentes** (idée du modèle en V) : chaque exigence reçoit ses critères d'acceptation, chaque cas d'utilisation ses scénarios, chaque transition d'état ses cas de test.

Phase de conception	Test préparé	Moment d'exécution
Analyse des besoins	Tests d'acceptation (critères de réussite)	Recette finale
Cahier des charges, cas d'utilisation	Tests fonctionnels / système	Phase 6
Conception technique (API, base)	Tests d'intégration	Pendant et après le développement
Code (fonctions, services)	Tests unitaires	Pendant le développement



À RETENIR

Règle d'or

Une exigence qui n'a aucun test associé est une exigence qui ne sera jamais vraiment validée. La matrice de traçabilité (chapitre 3) sert à le vérifier.

7.2 Les types de tests

Type	Ce qu'il vérifie	Exemple SupportIT	Outils courants
Unitaire	Une fonction ou une classe isolée.	transitionAutorisee('CLOS', 'OUVERT') renvoie faux.	Jest
Intégration	Collaboration entre modules (API + base).	POST /api/tickets enregistre une ligne dans tickets et dans l'historique.	Jest, Supertest
Fonctionnel	Une fonctionnalité répond à son exigence.	Un employé crée un ticket et le voit dans sa liste.	Scénarios manuels, Postman
Système	Le système complet dans un environnement proche du réel.	Parcours complet création → affectation → clôture.	Scénarios de bout en bout
Sécurité	Résistance aux accès non autorisés et aux attaques courantes.	Un employé ne peut pas lire le ticket d'un autre ; tentative d'injection SQL refusée.	Tests manuels, audit des dépendances
Performance	Temps de réponse et tenue en charge.	Liste de 5 000 tickets en moins de 2 s (BNF-03).	Outils de charge, mesures du navigateur
Acceptation	Le client valide que le système répond à son besoin.	Recette avec le responsable IT (fictif).	Procès-verbal de recette

7.3 Construire un plan de test

Aa

DÉFINITION

Plan de test

Document qui décrit **ce qui sera testé, comment, avec quelles données, par qui et dans quel environnement**, ainsi que les critères de réussite.

Rubrique	Contenu pour SupportIT
Objectif et périmètre	Vérifier les exigences EX-01 à EX-11 ; hors périmètre : charge supérieure à 5 000 tickets.
Stratégie	Unitaires et intégration automatisés ; fonctionnels et acceptation manuels.
Environnement	Base MySQL de test séparée, API en local, navigateur récent.
Données de test	Un compte par rôle, 3 catégories, un jeu de tickets dans chaque statut.
Responsabilités	Étudiant (exécution), encadrant (revue), responsable IT fictif (acceptation).
Critères d'entrée	Version taguée, schéma chargé, exigences gelées.
Critères de sortie	Toutes les exigences « Must » conformes ; aucune anomalie bloquante ouverte.
Planning	Voir semaines 17 à 21 du planning.

7.4 Scénarios et cas de test

Un **scénario** décrit une situation d'usage ; un **cas de test** en est une instance précise : données d'entrée, résultat attendu. Pour chaque exigence, testez le cas nominal, les cas limites et les cas d'erreur.

Exemple de tableau de test (SupportIT — résultats pédagogiques fictifs)

ID	Fonction-nalité	Scénario	Données d'entrée	Résultat attendu	Résultat obtenu	Statut
TC-01	Connexion	Identifiants valides	email et mot de passe d'un employé	200 + jeton ; accès au tableau de bord	Conforme	OK
TC-02	Connexion	Mot de passe incorrect	email valide, mot de passe faux	401 « Identifiants invalides »	Conforme	OK
TC-03	Créer ticket	Création nominale	titre 12 car., description, catégorie 1	201 ; ticket au statut OUVERT	Conforme	OK
TC-04	Créer ticket	Titre trop court	titre « abc »	400 + message de validation	Conforme	OK
TC-05	Affectation	Affecter par l'administrateur	ticket 7, technicien 3	Statut AFFECTE ; historique créé	Conforme	OK
TC-06	Statut	Transition autorisée	EN_COURS → RESOLU	200 ; historique complété	Conforme	OK
TC-07	Statut	Transition interdite	CLOS → OUVERT	409 ou 400 ; statut inchangé	Statut modifié (anomalie BUG-004)	KO
TC-08	Rôles	Employé accède à l'administration	jeton EMPLOYE sur GET /api/utilisateurs	403 Accès interdit	Conforme	OK

Dans un vrai projet, la colonne « Résultat obtenu » est remplie lors de l'exécution, avec la date et la version testée. Le cas TC-07 illustre une anomalie découverte et traitée dans la section suivante.

7.5 Automatiser : exemples de tests

JEST ET SUPERTEST

```
// tests/transitions.test.js  (test unitaire)
const { transitionAutorisee } = require('../src/services/ticketService');

test('EN_COURS peut passer à RESOLU', () => {
  expect(transitionAutorisee('EN_COURS', 'RESOLU')).toBe(true);
});
test('CLOS ne peut pas revenir à OUVERT', () => {
  expect(transitionAutorisee('CLOS', 'OUVERT')).toBe(false);
});

// tests/tickets.test.js  (test d'intégration, jeton généré dans la configuration des tests)
const request = require('supertest');
const app = require('../src/app');

describe('POST /api/tickets', () => {
  it('refuse une requête sans jeton', async () => {
    const res = await request(app).post('/api/tickets').send({ titre: 'Panne réseau' });
    expect(res.status).toBe(401);
  });
  it('refuse un titre trop court', async () => {
    const res = await request(app).post('/api/tickets')
      .set('Authorization', `Bearer ${jetonEmploye}`)
      .send({ titre: 'abc', description: 'Écran noir', categorieId: 1 });
    expect(res.status).toBe(400);
  });
});
```

Lancez `npm test` à chaque modification et conservez les résultats ; pensez aux cas limites (champ vide, jeton expiré, double clic).

7.6 Identifier, corriger et suivre les bugs

1. **Reproduire**
   Notez les étapes exactes, les données, l'environnement et la version.
2. **Documenter**
   Fiche d'anomalie : identifiant, description, résultat obtenu / attendu, gravité.
3. **Analyser**
   Remonter à la cause : erreur de code, de conception ou d'exigence ? Dans ce dernier cas, la correction passe par la phase concernée.
4. **Corriger**
   Branche dédiée, correction minimale, test qui échouait avant et réussit après.
5. **Retester**
   Rejouer le cas en échec et les cas voisins (régression).

Gravité	Définition	Exemple
Bloquante	Empêche d'utiliser une fonction essentielle ou compromet la sécurité.	Un employé accède aux tickets de tous.
Majeure	Fonction dégradée, contournement difficile.	Transition interdite acceptée (BUG-004).

Gravité	Définition	Exemple
Mineure	Gêne sans impact fonctionnel.	Message d'erreur mal formulé.
Cosmétique	Défaut d'affichage.	Alignement d'un bouton.

FICHE D'ANOMALIE — EXEMPLE (BUG-004, cas fictif)

Titre : Un ticket CLOS peut repasser à OUVERT — **Gravité :** majeure — **Version :** 0.9.2 — **Lié à :** TC-07, EX-06

Étapes : 1) se connecter comme administrateur ; 2) ouvrir un ticket CLOS ; 3) envoyer PATCH /statut avec « OUVERT ».

Attendu : refus (409) — **Obtenu :** 200 et statut OUVERT.

Cause : la route n'appelait pas transitionAutorisee. **Correction :** contrôle ajouté dans ticketService ; tests TC-07 et unitaire ajoutés. **Statut :** corrigé, retesté (version 0.9.3).

7.7 Documenter les résultats et valider

Le **rapport de tests et validation** est le livrable de la phase 6. Il prouve, pièces à l'appui, que les exigences sont satisfaites.

Section	Contenu
1. Introduction	Objet, version testée, environnement.
2. Stratégie	Types de tests, outils, données.
3. Résultats	Tableau des cas (OK / KO), synthèse chiffrée par exigence.
4. Anomalies	Liste, gravité, état de correction.
5. Matrice de traçabilité	Exigences ↔ tests ↔ résultats.
6. Recette	Procès-verbal d'acceptation signé ou validé par écrit.
7. Conclusion	Risques résiduels, réserves, perspectives.

VALIDER UNE FONCTIONNALITÉ (CRITÈRES « TERMINÉ »)	
QUALITÉ	
☐	Le code est relu et fusionné dans develop
☐	Les tests unitaires et d'intégration passent
☐	Les cas de test fonctionnels sont exécutés et documentés
CONFORMITÉ	
☐	Les critères d'acceptation de l'exigence sont satisfaits
☐	Aucune anomalie bloquante ou majeure ouverte
☐	Documentation mise à jour (API, guide utilisateur)

À RETENIR

Ce que vous retenez

Préparez les tests dès la conception et reliez-les aux exigences.

Un cas de test précise des données et un résultat attendu vérifiable.

Une anomalie se documente, se corrige, se retest ; la recette est formalisée par écrit.

CHAPITRE 08

Phase 7 : déploiement et livraison

08

Mettre l'application en ligne de façon sûre : serveur, variables d'environnement, hébergement, base de données, sauvegardes, vérification finale.

AU PROGRAMME

8.1 Préparer l'environnement de production

8.2 Choisir une solution d'hébergement

8.3 Configurer le serveur

8.4 Déployer le back-end, le front-end et la base

8.5 Sécurité de production et sauvegardes

8.6 Vérification finale et livrables

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : application déployée et documentation technique (V7) · Toolbox : modèle n°12

8.1 Préparer l'environnement de production

L'environnement de **production** est celui que les vrais utilisateurs utilisent. Il diffère du développement : HTTPS, secrets différents, base propre (sans données de test), journalisation, sauvegardes.

Environnement	Usage	Base de données	Secrets
Développement	Écrire et tester le code.	Locale, données de test.	Valeurs de développement.
Test / recette	Valider avant livraison.	Copie contrôlée, données fictives.	Propres à cet environnement.
Production	Utilisation réelle.	Dédiée, sauvegardée.	Forts, uniques, hors dépôt.

AVANT DE DÉPLOYER	
CODE ET VERSION	
☐	Version taguée (v1.0.0) sur la branche main
☐	Tests automatisés passés, recette validée (V6)
CONFIGURATION	
☐	Variables d'environnement de production préparées
☐	Comptes de démonstration supprimés ou mots de passe changés
☐	Mode débogage désactivé

8.2 Choisir une solution d'hébergement

Plusieurs solutions conviennent à un projet étudiant. Les **offres, limites et gratuités changent régulièrement** : vérifiez les conditions actuelles de chaque fournisseur avant de décider, et demandez à l'entreprise si une infrastructure est déjà prévue.

Option	Pour qui / quoi	Avantages	Points d'attention
Serveur de l'entreprise ou de l'établissement	Cas le plus courant en PFE.	Aucun coût, règles de sécurité déjà cadrées.	Procédures d'accès, délais de mise à disposition.
VPS (serveur privé virtuel)	Front, back et base sur une même machine.	Contrôle total, bon pour apprendre.	Vous administrez la sécurité et les mises à jour.
Hébergement statique	Front-end React compilé (HTML, CSS, JS).	Simple, souvent HTTPS inclus.	Ne fait tourner que des fichiers statiques.
Plateforme PaaS	Back-end Node.js déployé depuis Git.	Déploiement rapide, moins d'administration.	Limites d'offre, mise en veille possible, dépendance au fournisseur.
Base de données managée	MySQL hébergé par un fournisseur.	Sauvegardes et maintenance facilitées.	Coût, latence, règles d'accès réseau.
Conteneurs (Docker)	Reproduire l'environnement partout.	Cohérence entre machines.	Un apprentissage supplémentaire.

CONSEIL PROFESSIONNEL

Pensez à la démonstration

Même si l'entreprise héberge la version « officielle », prévoyez une instance accessible pour la soutenance (ou une vidéo de secours) : la salle du jury n'a pas forcément accès à votre réseau d'entreprise.

8.3 Configurer le serveur

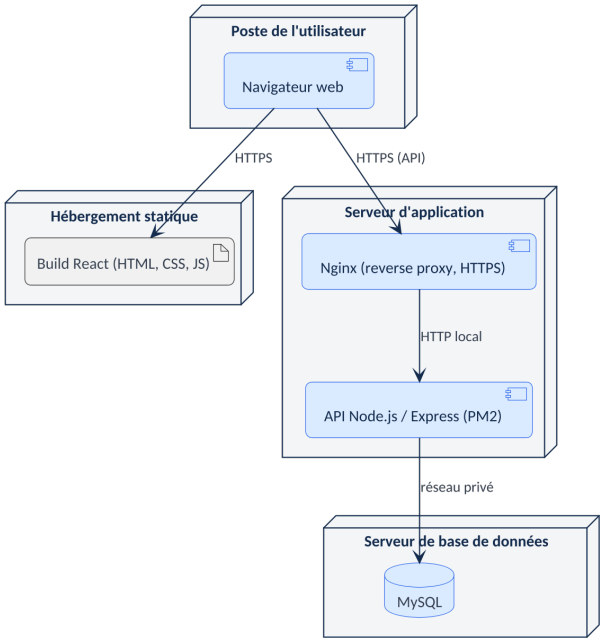


Figure 11 — Architecture de déploiement de SupportIT (exemple pédagogique).

Exemple de préparation d'un serveur Linux (VPS) : adaptez les commandes à votre distribution et à vos droits.

SERVEUR LINUX — PRÉPARATION (EXEMPLE)

```
# Pare-feu : n'ouvrir que le nécessaire
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable

# Outils : Node.js (version LTS), Nginx, MySQL, PM2
sudo apt update && sudo apt install -y nginx mysql-server
sudo npm install -g pm2

# Utilisateur applicatif sans privilèges d'administration
sudo adduser supportit
```

- Désactivez la connexion SSH par mot de passe au profit des clés, si possible.
- Le port MySQL (3306) ne doit **pas** être exposé sur Internet : l'API s'y connecte en local ou via un réseau privé.
- Placez les variables d'environnement dans un fichier lisible uniquement par l'utilisateur applicatif.

8.4 Déployer le back-end, le front-end et la base

Back-end

DÉPLOIEMENT DE L'API AVEC PM2

```
cd /srv/supportit/backend
git pull origin main
npm ci --omit=dev
cp .env.example .env        # puis renseigner les valeurs de production
pm2 start src/server.js --name supportit-api
pm2 save && pm2 startup
```

NGINX — REVERSE PROXY (EXEMPLE)

```
server {
  listen 80;
  server_name api.example.com;
  location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
# HTTPS : sudo certbot --nginx -d api.example.com   (certificat Let's Encrypt)
```

Front-end

Définissez l'adresse de l'API dans `frontend/.env.production` (`VITE_API_URL=https://api.example.com/api`), puis compilez : `npm run build`. Le dossier `dist/` est publié sur l'hébergement statique ou servi par Nginx. Mettez à jour `CORS_ORIGIN` côté API avec l'adresse du front.

Base de données

- Créer la base et l'utilisateur applicatif (droits limités) comme au chapitre 6.
- Charger `schema.sql` ; ne charger **aucun** jeu de données de test.
- Créer un premier compte administrateur avec un mot de passe fort, à changer à la première connexion.

8.5 Sécurité de production et sauvegardes

Mesure	Pourquoi
HTTPS partout	Protège identifiants et jetons en transit.
Secrets forts et uniques (JWT_SECRET, mot de passe BDD)	Une fuite ou une valeur devinable compromet tout le système.
Mises à jour du système et des dépendances	Corrige des failles connues.
Droits minimaux (BDD, utilisateur système)	Limite les dégâts en cas de compromission.
Limitation des tentatives de connexion, journaux sans données sensibles	Réduit la force brute et permet un diagnostic sûr.

Sauvegardes

Une sauvegarde qui n'a jamais été restaurée n'est pas une sauvegarde. Planifiez des exports réguliers, conservez-les hors du serveur (règle **3-2-1** : trois copies, deux supports, une copie hors site) et testez une restauration au moins une fois.

MYSQL — SAUVEGARDE (EXEMPLE)

```
# Export quotidien (mot de passe dans un fichier d'options protégé, pas sur la ligne de commande)
mysqldump --defaults-extra-file=~/.my.cnf --single-transaction supportit \
  > sauvegarde_$(date +%F).sql
# Planification (cron) : tous les jours à 2 h
0 2 * * * /srv/supportit/scripts/sauvegarde.sh
```

8.6 Vérification finale et livrables

VÉRIFICATION FINALE APRÈS DÉPLOIEMENT	
FONCTIONNEMENT	
☐	La page d'accueil se charge en HTTPS sans erreur dans la console
☐	Connexion réussie avec chaque rôle
☐	Création, affectation et changement de statut d'un ticket fonctionnent
☐	Le tableau de bord s'affiche
SÉCURITÉ	
☐	L'API refuse les requêtes sans jeton (401)
☐	Un employé n'accède pas aux fonctions d'administration (403)
☐	Aucun message d'erreur ne révèle de détails techniques
EXPLOITATION	
☐	Redémarrage du serveur : l'API repart seule
☐	Sauvegarde exécutée et restauration testée
☐	Journaux accessibles

Livrable de la phase 7 : application déployée, procès-verbal de livraison et documentation technique (adresses, procédure de déploiement, variables sans valeurs secrètes, architecture, sauvegarde et restauration).

À RETENIR

Ce que vous retenez

La production se prépare : secrets, HTTPS, base propre, droits minimaux.

Choisissez l'hébergement selon le contexte et vérifiez les conditions actuelles.

Vérifiez, sauvegardez, documentez : une livraison sans vérification n'est pas une livraison.

M.D.S

CHAPITRE 09

Documentation et rapport final

09

Documenter chaque phase, produire guides et documentation d'API, et relier le travail réalisé à la structure du rapport de PFE.

AU PROGRAMME

9.1 Les documents à produire

9.2 Documentation d'API et guide d'installation

9.3 Guide utilisateur, captures et diagrammes

9.4 Relier les phases au rapport de PFE

9.5 Constituer le dossier documentaire

LIVRABLE ET OUTILS ASSOCIÉS

Livrable : dossier documentaire complet · Toolbox : modèles n°10 et n°11

9.1 Les documents à produire

Dans la cascade, la documentation n'est pas une corvée finale : elle est produite **phase après phase** et constitue la mémoire du projet. Elle sert à valider, à développer, à maintenir et à rédiger le rapport.

Document	Public	Contenu	Phase source
Documentation fonctionnelle	Client, encadrants	Besoins, cahier des charges, cas d'utilisation, règles de gestion.	Phases 1 à 3
Documentation technique	Développeurs, successeurs	Architecture, modèle de données, choix techniques, conventions.	Phases 4 et 5
Documentation d'API	Développeurs front-end, intégrateurs	Routes, paramètres, réponses, erreurs, authentification.	Phases 4 et 5
Guide d'installation	Développeurs, administrateurs	Prérequis, installation, configuration, lancement, tests.	Phases 5 et 7
Guide utilisateur	Employés, techniciens, administrateurs	Utilisation pas à pas, captures d'écran, questions fréquentes.	Phases 6 et 7
Rapport de réalisation	Jury, encadrants	Synthèse du projet, démarche, résultats, bilan.	Toutes

9.2 Documentation d'API et guide d'installation

Documenter une route

POST /api/tickets — CRÉER UN TICKET

Accès : rôle EMPLOYE (jeton Bearer obligatoire) — **Exigence :** EX-03

Corps (JSON) : titre (texte, 5–150), description (texte), categorieId (entier), priorite (optionnel : BASSE, NORMALE, HAUTE, CRITIQUE)

Réponse 201 : { "id": 42, "titre": "...", "statut": "OUVERT", "priorite": "NORMALE", "dateCreation": "..." }

Erreurs : 400 (validation), 401 (jeton absent ou invalide), 403 (rôle non autorisé)

Exemple : curl -X POST .../api/tickets -H "Authorization: Bearer <jeton>" -H "Content-Type: application/json" -d '{"titre":"Panne réseau","description":"...","categorieId":1}'

Pour aller plus loin, la description peut être formalisée avec la spécification **OpenAPI** (outil Swagger), qui génère une documentation interactive.

Guide d'installation

1. Prérequis (versions de Node.js, MySQL).
2. Récupération du code (git clone).
3. Création de la base et chargement du schéma.
4. Configuration des fichiers .env à partir des modèles.

5. Installation des dépendances (npm install) et lancement du back-end puis du front-end.
6. Vérification : adresse de l'application, compte de démonstration, commande de tests.

9.3 Guide utilisateur, captures et diagrammes

- **Plan du guide utilisateur :** connexion ; créer un ticket ; suivre mes tickets ; (technicien) traiter un ticket ; (administrateur) gérer les comptes, affecter, consulter le tableau de bord ; problèmes fréquents.
- **Captures d'écran :** nettes, recadrées, avec des données fictives ; une légende numérotée et une phrase d'explication ; cohérentes avec la version livrée.
- **Diagrammes :** conservez les sources (.puml) et les images exportées ; citez l'identifiant de l'exigence concernée ; vérifiez qu'ils correspondent au code final.
- **Annexes :** comptes rendus de réunion, cas de test détaillés, scripts SQL, extraits de code volumineux, guides complets.



ATTENTION

Documentation et réalité

Une documentation qui ne correspond plus au code est pire qu'une absence de documentation. Relisez-la à chaque version livrée et mentionnez la version concernée.

9.4 Relier les phases au rapport de PFE

Le rapport n'est pas à écrire de zéro à la fin : chaque phase fournit la matière d'un chapitre. Votre établissement impose son propre plan ; voici une correspondance usuelle.

Phase de la cascade	Livrable	Partie du rapport
1. Analyse des besoins	Document d'analyse	Introduction ; présentation de l'entreprise ; étude de l'existant ; problématique
2. Cahier des charges	Cahier des charges	Analyse des besoins (besoins fonctionnels et non fonctionnels, périmètre)
3. Conception fonctionnelle	Diagrammes UML, spécifications	Chapitre de conception (modélisation fonctionnelle)
4. Conception technique	Dossier de conception technique	Chapitre de conception (architecture, base de données, choix technologiques)
5. Développement	Code, README	Chapitre de réalisation (environnement, implémentation, interfaces)
6. Tests	Rapport de tests	Chapitre de tests et validation
7. Déploiement	Application et documentation	Réalisation (déploiement) ; conclusion et perspectives
Transversal	Journal de décisions, comptes rendus	Annexes ; bibliographie ; remerciements

À VÉRIFIER AUPRÈS DE VOTRE ÉTABLISSEMENT

Plan et exigences du rapport

Le plan, la longueur, la mise en page, la langue et le style de citation sont fixés par votre établissement. Cette correspondance est un guide pour réutiliser vos livrables, pas un plan officiel.

9.5 Constituer le dossier documentaire

DOSSIER DOCUMENTAIRE COMPLET	
DOSSIERS DE PHASE	
☐	Document d'analyse des besoins (V1 validée)
☐	Cahier des charges et spécifications (V2)
☐	Dossier de conception fonctionnelle et UML (V3)
☐	Dossier de conception technique (V4)
☐	Rapport de tests et validation (V6)
☐	Documentation de déploiement et procès-verbal de livraison (V7)
DOCUMENTS D'USAGE	
☐	README et guide d'installation
☐	Documentation d'API
☐	Guide utilisateur illustré
PIÈCES DE SUIVI	
☐	Matrice de traçabilité à jour
☐	Comptes rendus et journal des décisions
☐	Planning réel et historique des versions

À RETENIR

Ce que vous retenez

Chaque phase produit un document qui nourrit le rapport.

La documentation doit rester synchronisée avec le code livré.

Un dossier documentaire complet prouve la rigueur de la démarche en cascade.

M.D.S

CHAPITRE 10

Gestion du projet et conseils professionnels

10

Organiser son temps, planifier les phases, travailler avec son encadrant, gérer imprévus et changements, suivre l'avancement et préparer la démonstration finale.

AU PROGRAMME

10.1 Organiser son temps

10.2 Planning global d'un PFE en cascade

10.3 Travailler avec un encadrant

10.4 Gérer les imprévus et les changements

10.5 Suivre l'avancement et documenter les décisions

10.6 Présenter les livrables et la démonstration finale

10.7 Les 20 erreurs à éviter

10.8 Se démarquer en tant que futur ingénieur logiciel

LIVRABLE ET OUTILS ASSOCIÉS

Planning prévisionnel sur 4 à 6 mois · Toolbox : modèle n°5 (planning) et n°12 (checklist de livraison)

10.1 Organiser son temps

- **Fixez les dates imposées** d'abord (dépôt, soutenance), puis planifiez à rebours.
- **Réservez des plages protégées** pour le travail profond (analyse, conception, code) et d'autres pour les échanges.
- **Terminez les phases** : un document 80 % fini qui bloque la phase suivante coûte plus que quelques jours supplémentaires.
- **Gardez une marge** (souvent 15 à 20 % du temps, repère personnel) pour l'imprévu.
- **Travaillez le rapport en continu** plutôt qu'en fin de projet.

CONSEIL PROFESSIONNEL

Une semaine type

Exemple : lundi planification et point encadrant ; mardi à jeudi travail de la phase en cours ; vendredi documentation, mise à jour du suivi et sauvegarde. Adaptez aux consignes de l'entreprise.

10.2 Planning global d'un PFE en cascade

Voici un **exemple illustratif** sur 24 semaines (six mois). Les durées dépendent de la complexité du projet et des contraintes de l'établissement ; ce planning n'est pas une obligation universelle.

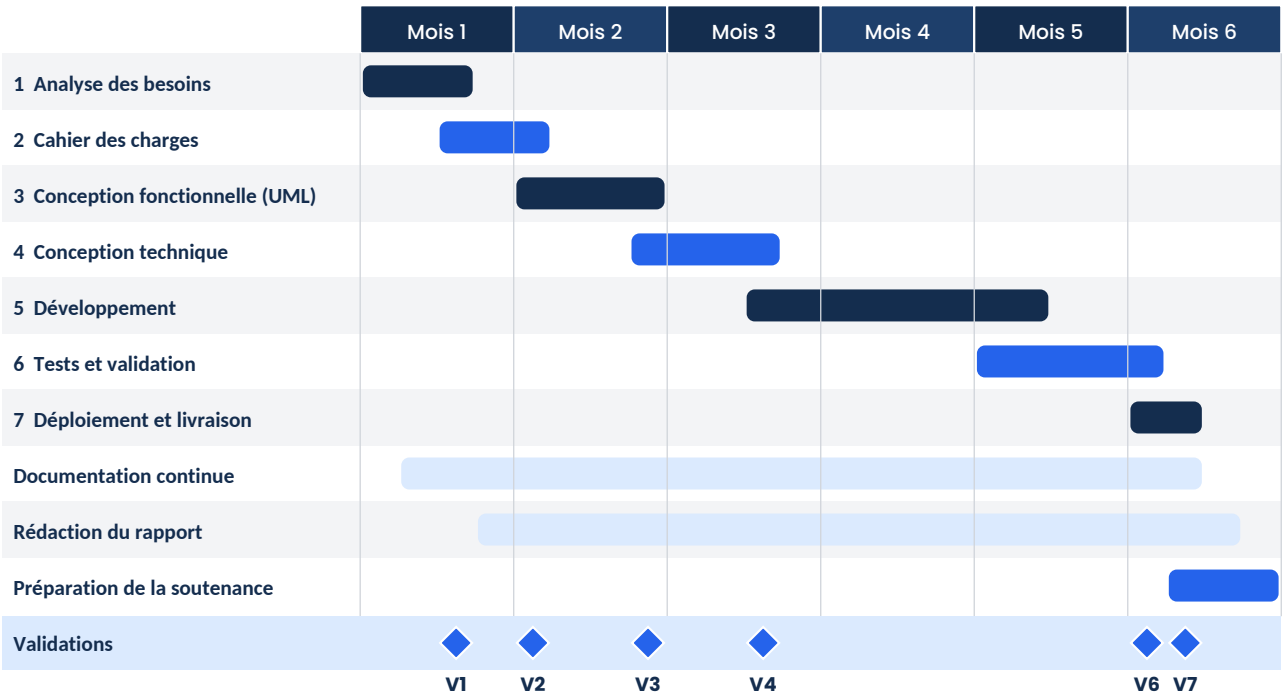


Figure 12 — Diagramme de Gantt illustratif d'un PFE en cascade sur 24 semaines (V = validations).

Phase	Durée (6 mois)	Durée (4 mois)	Activités principales	Livrable et validation
1. Analyse des besoins	3 sem.	2 sem.	Entretiens, étude de l'existant, faisabilité.	Document d'analyse (V1)

Phase	Durée (6 mois)	Durée (4 mois)	Activités principales	Livrable et validation
2. Cahier des charges	2–3 sem.	2 sem.	Exigences, priorisation, planning.	Cahier des charges (V2)
3. Conception fonctionnelle	3 sem.	2 sem.	UML, spécifications, maquettes.	Dossier fonctionnel (V3)
4. Conception technique	3 sem.	2 sem.	Architecture, BDD, API, sécurité.	Dossier technique (V4)
5. Développement	7 sem.	5 sem.	Environnement, back-end, front-end, intégration.	Version candidate (V5)
6. Tests et validation	4 sem.	2 sem.	Plan de test, exécution, corrections, recette.	Rapport de tests (V6)
7. Déploiement	1–2 sem.	1 sem.	Mise en production, vérification, documentation.	Livraison (V7)
Soutenance	2–3 sem. (chevauche)	1–2 sem.	Rapport final, présentation, démonstration.	Dépôt et soutenance

En quatre mois, on comprime les phases et on réduit le périmètre (priorités « Could » écartées) ; on ne supprime ni les validations ni les tests. Les tests unitaires s'écrivent en continu pendant le développement.

10.3 Travailler avec un encadrant

- **Préparez chaque point :** ce qui est fait, ce qui bloque, ce que vous demandez, les décisions à prendre.
- **Envoyez les documents à l'avance** avant une validation (au moins quelques jours).
- **Consignez par écrit** décisions et remarques (compte rendu).
- **Distinguez** encadrant académique (exigences universitaires, rapport) et encadrant professionnel (besoin métier, technique).
- **Soyez transparent** sur les retards : proposez une solution (réduire le périmètre, décaler un jalon).
- **Remerciez et valorisez** le temps qui vous est accordé.

10.4 Gérer les imprévus et les changements

Dans la cascade, le changement n'est pas interdit : il est **géré**. Quand un besoin évolue après validation, on suit une procédure de demande de modification, pour décider en connaissance de cause.

1. **Enregistrer la demande**
   Qui, quoi, pourquoi, urgence.
2. **Analyser l'impact**
   Quels documents, quelles phases, quels tests, quelle durée ?
3. **Décider**
   Accepter, reporter ou refuser, avec l'encadrant ; arbitrer sur le périmètre ou le planning.
4. **Mettre à jour**
   Documents concernés (nouvelle version), matrice de traçabilité, planning.

5 **Revalider**
Les parties modifiées sont relues et validées à nouveau.

Risque (exemples)	Probabilité / impact	Mesure
Besoin mal compris, découvert en recette	Moyenne / élevé	Validations V1 à V3 ; maquettes ; prototypes précoces.
Retard de développement	Moyenne / élevé	Priorisation MoSCoW ; marge ; point hebdomadaire.
Indisponibilité du client ou de l'encadrant	Moyenne / moyen	Calendrier de validations fixé à l'avance.
Perte de données ou de code	Faible / très élevé	Git distant, sauvegardes, règle 3-2-1.
Technologie mal maîtrisée	Moyenne / moyen	Petit prototype technique pendant la conception.
Accès à l'environnement de production retardé	Moyenne / moyen	Instance de démonstration de secours.

10.5 Suivre l'avancement et documenter les décisions

Phase	Statut	Avancement	Validation	Remarque
1. Analyse	Terminée	100 %	V1 signée	—
2. Cahier des charges	Terminée	100 %	V2 signée	Version 1.1
3. Conception fonctionnelle	En cours	70 %	V3 prévue	Séquences à finaliser
4. Conception technique	À venir	0 %	—	—

*Exemple de tableau de suivi : il se met à jour chaque semaine et se présente à l'encadrant. L'avancement se mesure sur des **livrables validés**, pas sur des impressions.*

JOURNAL DES DÉCISIONS — EXEMPLE

D-007 — 12/03 : choix de MySQL plutôt que MongoDB. **Contexte :** données très relationnelles, transactions utiles. **Alternatives :** MongoDB, PostgreSQL. **Décision prise avec :** encadrant professionnel. **Conséquence :** schéma relationnel, requêtes SQL préparées.

D-011 — 02/04 : application mobile exclue de cette version. **Raison :** périmètre et délai. **Suite :** inscrite en perspectives.

CONSEIL PROFESSIONNEL
Pourquoi un journal de décisions ?
Il évite de rejouer des débats, prouve votre démarche au jury et fournit directement des paragraphes pour le rapport (« nous avons choisi X parce que… »).

10.6 Présenter les livrables et la démonstration finale

- **Livrer proprement :** un dossier par phase, nommé et versionné, avec une page de garde et l'historique des versions.
- **Présenter les validations :** à chaque jalon, montrez le livrable, les remarques traitées et l'accord obtenu.
- **Préparer la démonstration :** scénario court qui suit un fil rouge (un employé crée un ticket → l'administrateur l'affecte → le technicien le traite → le tableau de bord se met à jour).
- **Plan B :** vidéo ou captures de la démonstration en cas d'incident ; comptes et données de démonstration prêts.
- **Relier au cahier des charges :** montrez quelles exigences sont couvertes, lesquelles sont en perspective.

CHECKLIST DE LA DÉMONSTRATION FINALE	
AVANT	
☐	Scénario de 4 à 6 minutes écrit et répété
☐	Comptes (admin, technicien, employé) et données fictives prêts
☐	Application déployée testée le jour même
☐	Vidéo de secours enregistrée
PENDANT	
☐	Annoncer l'objectif de la démonstration
☐	Commenter chaque action
☐	Montrer un cas d'erreur géré (jeton expiré, champ invalide)
☐	Relier à l'exigence concernée

10.7 Les 20 erreurs à éviter

N°	Erreur	À faire plutôt
1	Commencer à coder sans avoir compris le besoin.	Valider l'analyse (V1) d'abord.
2	Rédiger un cahier des charges vague.	Exigences numérotées et vérifiables.
3	Ne pas faire valider les documents par écrit.	Trace datée de chaque validation.
4	Choisir une technologie par effet de mode.	Justifier par des critères.
5	Mélanger trop de technologies.	Une stack cohérente.
6	Diagrammes UML décoratifs, incohérents avec le code.	Mettre à jour à chaque changement.
7	Un seul gros diagramme pour tout.	Un diagramme par vue ou scénario.
8	Oublier la sécurité jusqu'à la fin.	Sécurité dès la conception.
9	Laisser des secrets dans Git.	.env ignoré, secrets en variables.
10	Travailler sur la branche principale.	Branches et pull requests.
11	Commits rares et énormes.	Petits commits clairs.

N°	Erreur	À faire plutôt
12	Écrire les tests à la dernière minute.	Préparer les cas dès la conception.
13	Tester uniquement le cas nominal.	Cas limites et erreurs.
14	Ne pas documenter les bugs.	Fiches d'anomalie.
15	Accepter tout changement sans analyse.	Procédure de modification.
16	Ne pas gérer le périmètre.	MoSCoW, liste « hors périmètre ».
17	Planning sans marge.	15 à 20 % de réserve.
18	Rédiger le rapport à la fin.	Écrire phase après phase.
19	Négliger les sauvegardes.	Dépôt distant et copies testées.
20	Démonstration non préparée.	Scénario, données, plan B.

10.8 Se démarquer en tant que futur ingénieur logiciel

Levier	Comment
Rigueur	Documents propres, versionnés, traçabilité à jour : c'est ce qui distingue un projet d'ingénieur d'un simple programme.
Compréhension du besoin	Posez les bonnes questions, reformulez, validez : les entreprises recherchent des profils qui comprennent leur métier.
Qualité du code	Architecture claire, tests, gestion d'erreurs, README : montrez que votre code est maintenable.
Sécurité et fiabilité	Rôles, validation, sauvegardes : peu d'étudiants y pensent.
Communication	Comptes rendus, présentations claires, transparence sur les risques.
Esprit critique	Présentez les limites de votre solution et les évolutions possibles.
Apprentissage continu	Veille régulière, documentation officielle, curiosité sur les méthodes agiles : savoir *quand* utiliser chaque méthode est un atout.

À RETENIR

Ce que vous retenez

Planifiez les phases et gardez des marges ; avancez sur des livrables validés.

Gérez les changements par une procédure, pas par l'improvisation.

Documentez vos décisions : elles nourrissent le rapport et rassurent le jury.

SECTION SPÉCIALE

Douze prompts IA pour les phases du projet

Des instructions prêtes à copier-coller pour utiliser l'IA comme assistant, pas comme remplaçant.

ATTENTION

Avant de commencer : vérifier et protéger

Vérifiez toujours les résultats : une IA peut produire des erreurs convaincantes (fausses références, code incorrect, exigences incohérentes). Comparez avec la documentation officielle, exécutez le code, relisez les diagrammes.

Ne transmettez jamais de données confidentielles de votre entreprise (code, base de données, documents internes, noms de clients) à un outil d'IA non autorisé. Anonymisez et demandez l'accord de l'entreprise et de votre établissement.

Respectez les règles de votre établissement sur l'usage de l'IA et citez-le si on vous le demande. Le travail rendu doit être le vôtre : vous devez le comprendre et pouvoir l'expliquer.

Remplacez les éléments entre crochets. Un bon prompt précise un **rôle**, un **contexte**, une **tâche** et un **format de réponse** ; demandez à l'IA de poser des questions et de critiquer plutôt que de produire un livrable que vous ne comprendriez pas.

PROMPT 1 Analyser les besoins d'un projet

```
Agis comme un analyste fonctionnel. Contexte : [description anonymisée du problème et de
l'entreprise]. À partir de ces informations :
1) liste les questions à poser aux utilisateurs et au client ;
2) propose des besoins fonctionnels (BF-xx) et non fonctionnels (BNF-xx) vérifiables ;
3) signale les ambiguïtés et les hypothèses à valider.
Ne suppose rien qui ne soit pas dans le texte : pose-moi des questions.
```

PROMPT 2 Construire un cahier des charges

```
Voici mon document d'analyse des besoins : [coller]. Génère un cahier des charges en 12 sections
(présentation, contexte, problématique, objectifs, étude de l'existant, solution proposée,
besoins fonctionnels, besoins non fonctionnels, contraintes techniques, périmètre, planning
prévisionnel, critères d'acceptation). Pour chaque exigence, propose un identifiant, une priorité
MoSCoW et un critère d'acceptation. Indique ce qui manque.
```

PROMPT 3 Identifier les acteurs

```
Contexte : [description du système]. Identifie les acteurs (humains et systèmes externes), leurs
objectifs, et propose 3 personas fictifs avec contexte, objectifs et difficultés. Présente le
résultat dans un tableau. Signale les acteurs que j'ai peut-être oubliés.
```

PROMPT 4 Générer des diagrammes PlantUML

```
Écris le code PlantUML d'un diagramme [de cas d'utilisation / de classes / de séquence] pour le
scénario suivant : [description]. Contraintes : noms en français, code syntaxiquement valide, pas
d'éléments non demandés. Après le code, liste les hypothèses faites et les points à vérifier.
```

PROMPT 5 Proposer une architecture technique

```
Besoins : [exigences principales]. Contraintes : [technologies imposées, durée, compétences].
Propose deux architectures cohérentes (une seule stack par proposition) avec : schéma textuel,
justification, avantages, limites et risques. Recommande-en une en expliquant selon quels
critères. Ne mélange pas plusieurs bases de données ou frameworks front-end.
```

PROMPT 6 Modéliser une base de données

```
Voici mon diagramme de classes en PlantUML : [coller]. Propose le modèle entité-association puis
le schéma relationnel pour MySQL (tables, clés primaires et étrangères, contraintes, index
utiles), et le script SQL de création. Explique chaque règle de passage utilisée et signale les
points de normalisation.
```

PROMPT 7 Préparer des scénarios de tests

```
Voici mes exigences et leurs critères d'acceptation : [coller]. Génère un tableau de cas de test
avec : ID, fonctionnalité, scénario, données d'entrée, résultat attendu. Couvre les cas nominaux,
limites et d'erreur, y compris les droits d'accès par rôle. Indique l'exigence couverte par
chaque cas.
```

PROMPT 8 Documenter une API

```
Voici le code d'une route (anonymisé) : [coller]. Rédige la documentation correspondante :
méthode et URL, rôle autorisé, paramètres, corps de requête, réponses (succès et erreurs avec
codes HTTP), exemple de requête curl. Ne documente que ce que le code fait réellement.
```

PROMPT 9 Analyser des erreurs de code

```
Environnement : [langage, version, framework]. Message d'erreur complet : [coller]. Code concerné
(anonymisé, sans secrets) : [coller]. Explique la cause probable, propose trois pistes de
diagnostic classées par probabilité et la correction. Indique comment vérifier que le problème
est résolu.
```

PROMPT 10 Préparer un plan de déploiement

```
Application : [description de la stack]. Cible : [type d'hébergement]. Propose un plan de
déploiement : prérequis, étapes, variables d'environnement à définir (sans valeurs), sécurité,
sauvegardes, vérifications finales et plan de retour arrière. Signale ce que je dois vérifier
dans la documentation de l'hébergeur.
```

PROMPT 11 Vérifier la qualité d'un rapport

```
Voici un extrait de mon rapport de PFE : [coller un chapitre, pas le rapport entier
confidentiel]. Évalue : clarté, cohérence avec le plan, définition des termes, argumentation des
choix, orthographe. Liste les problèmes par ordre d'importance avec des suggestions. Ne réécris
pas à ma place : explique-moi comment améliorer.
```

PROMPT 12 Préparer une démonstration

```
Mon projet : [résumé]. Durée de la démonstration : [x] minutes. Propose un scénario pas à pas qui
montre la valeur du système, les cas d'erreur à montrer, les risques techniques et un plan B.
Fournis aussi 10 questions que le jury pourrait poser sur la démonstration.
```

BOÎTE À OUTILS

Boîte à outils : douze modèles réutilisables

Documents et checklists à adapter à votre projet. Copiez le texte dans votre outil habituel (Word, Google Docs, tableur) ou imprimez-les.

1 Modèle de document d'analyse des besoins

Livrable de la phase 1 — validation V1

Section	À renseigner
Version et historique	V1.0 — [date] — [auteur] — [validateur]
Contexte	[Entreprise, activité, situation actuelle, enjeux]
Problématique	[Constat → problème → question centrale]
Parties prenantes et acteurs	[Rôle, besoins, personas]
Recueil des besoins	[Méthodes, dates, participants — comptes rendus en annexe]
Objectifs et périmètre	Général : [...] Spécifiques : OS1 [...] OS2 [...] Inclus : [...] Exclus : [...]
Besoins	BF-01 [...] BNF-01 [...] (vérifiables)
Contraintes et faisabilité	[Technique, organisationnelle, temporelle, économique, juridique]
Risques	[Risque — probabilité — impact — mesure]
Validation	[Nom, rôle, date, signature]

2 Modèle de cahier des charges

Livrable de la phase 2 — validation V2

Section	À renseigner
1. Présentation générale	[Projet, auteur, version, destinataires]
2. Contexte du projet	[...]
3. Problématique	[...]
4. Objectifs	[Généraux et spécifiques (SMART)]
5. Étude de l'existant	[Processus actuel, outils, limites, solutions étudiées]
6. Solution proposée	[Description globale]
7. Besoins fonctionnels	EX-01 [...] (Must/Should/Could)
8. Besoins non fonctionnels	ENF-01 [...] — critère mesurable
9. Contraintes techniques	[Technologies, environnement, sécurité]
10. Périmètre	Inclus [...] Exclu [...]
11. Planning prévisionnel	[Phases, dates, jalons, livrables]
12. Critères d'acceptation	[Pour chaque exigence : condition de validation]

3 Modèle de spécifications fonctionnelles

Une fiche par cas d'utilisation important — validation V3

Rubrique	À renseigner
Identifiant et nom	UC-[xx] — [Verbe + complément] — exigences liées : [EX-xx]
Acteur principal / secondaires	[...]
Préconditions	[...]
Scénario nominal	1. [...] 2. [...] 3. [...]
Scénarios alternatifs et erreurs	[2a. ... 3b. ...]
Postconditions	[...]
Règles de gestion	RG-[xx] [...]
Maquettes / écrans	[Références]
Critères d'acceptation	[...]

4 Modèle de spécifications techniques

Dossier de conception technique — validation V4

Section	À renseigner
Architecture	[Style, schéma, justification]
Stack technologique	Front [...] Back [...] BDD [...] Auth [...] — critères de choix [...]
Modèle de données	[ER, schéma relationnel, script SQL]
API	[Tableau : méthode, route, rôle, exigence]
Sécurité	[Authentification, rôles, validation, secrets, HTTPS]
Organisation du code	[Arborescence, conventions de nommage]
Environnements et déploiement	[Développement, test, production, hébergement]
Qualité et tests	[Stratégie, outils]

5 Modèle de planning prévisionnel

À compléter avec vos dates réelles (4 à 6 mois)

Phase	Début	Fin	Activités	Livrable	Valid.

6 Exemple de diagramme de cas d'utilisation

Squelette PlantUML à adapter (exemple complet : chapitre 4)

PLANTUML — SQUELETTE

```
@startuml
left to right direction
actor "Acteur A" as A
actor "Acteur B" as B
rectangle "Nom du système" {
  usecase "Objectif 1" as UC1
  usecase "Objectif 2" as UC2
  usecase "S'authentifier" as UC0
}
A --> UC1
B --> UC2
UC1 ..> UC0 : <<include>>
@enduml
```

7 Exemple de diagramme de classes

Squelette PlantUML à adapter

PLANTUML — SQUELETTE

```
@startuml
hide empty members
class Entite1 {
  - id : int
  - attribut : String
  + operation() : void
}
class Entite2 {
  - id : int
}
enum Etat {
  ETAT_A
  ETAT_B
}
Entite1 "1" -- "0..*" Entite2 : association >
Entite1 ..> Etat
@enduml
```

8 Exemple de diagramme de séquence

Squelette PlantUML à adapter

PLANTUML — SQUELETTE

```
@startuml
actor Utilisateur as U
participant "Interface" as UI
participant "API" as API
database "Base de données" as DB
U -> UI : action
UI -> API : requête (jeton)
alt cas nominal
  API -> DB : opération
  DB --> API : résultat
  API --> UI : 200 OK
else cas d'erreur
  API --> UI : 4xx
end
@enduml
```

9 Modèle de plan de test

Tableau des cas de test à compléter lors de la phase 6

ID	Fonctionnalité	Scénario	Données d'entrée	Résultat attendu	Résultat obtenu	Statut

10 Modèle de documentation API

Une fiche par route

Rubrique	À renseigner
Méthode et URL	[GET | POST | PATCH | DELETE] /api/[ressource]
Description et exigence	[Rôle de la route] — [EX-xx]
Accès	[Public | rôle] — authentification [Bearer]
Paramètres (URL, requête)	[nom — type — obligatoire — description]
Corps de requête	[champs JSON, types, contraintes]
Réponses	[code — corps JSON — signification]
Erreurs	[400, 401, 403, 404, 409 … — messages]
Exemple	[commande curl et réponse]

11 Modèle de README GitHub

À placer à la racine du dépôt

README.MD — MODÈLE

```
# [Nom du projet]
[Description en deux phrases : problème résolu, public visé.]

## Fonctionnalités
- [Fonctionnalité 1]
- [Fonctionnalité 2]

## Technologies
[Front-end, back-end, base de données, outils]

## Prérequis
[Node.js x.y, MySQL x.y, Git]

## Installation
1. git clone [url]
2. Créer la base et charger database/schema.sql
3. Copier .env.example en .env et renseigner les valeurs
4. npm install (backend et frontend)

## Lancement et tests
npm run dev  |  npm test

## Structure du projet
[Arborescence commentée]

## Documentation
[Liens vers docs/ : analyse, cahier des charges, UML, API]

## Auteur et licence
[Nom] — projet de fin d'études, [établissement], [année]
```

12 Checklist de livraison du projet

À valider avant la remise finale

LIVRAISON	
DOCUMENTS	
☐	Analyse, cahier des charges, conception fonctionnelle et technique validés (V1 à V4)
☐	Matrice de traçabilité à jour
☐	Rapport de tests et PV de recette
☐	Documentation d'API, guide d'installation, guide utilisateur
LOGICIEL	
☐	Code sur la branche main, version taguée
☐	Aucun secret dans le dépôt
☐	Application déployée et vérifiée
☐	Sauvegarde réalisée et restauration testée
SOUTENANCE	
☐	Rapport conforme au gabarit, relu et exporté en PDF
☐	Démonstration scénarisée avec plan B
☐	Présentation chronométrée

ANNEXES

Annexes et bibliographie

Glossaire, bloc de style PlantUML, références.

A Glossaire

Terme	Définition
API REST	Interface qui expose des ressources via les verbes HTTP, généralement en JSON.
Cahier des charges	Document décrivant exigences, contraintes, périmètre et critères d'acceptation.
Cas d'utilisation	Objectif qu'un acteur atteint avec le système.
Exigence	Besoin reformulé de façon précise, vérifiable et numérotée.
Jalon / validation	Point de contrôle où un livrable est revu et accepté avant la phase suivante.
JWT	Jeton signé (RFC 7519) utilisé pour authentifier les requêtes.
Livrable	Document ou logiciel produit à la fin d'une phase.
MoSCoW	Méthode de priorisation : Must, Should, Could, Won't.
Persona	Utilisateur type fictif utilisé pour concevoir.
Recette	Validation finale par le client sur la base de critères d'acceptation.
Traçabilité	Lien suivi entre besoin, exigence, conception, code et test.
UML	Langage graphique normalisé de modélisation.
Waterfall	Modèle en cascade : phases séquentielles validées.

B Bloc de style PlantUML utilisé dans ce guide

Les diagrammes de ce guide utilisent un fichier de style commun (`skin.iuml`) inclus avec `!include skin.iuml`. Il est facultatif : sans lui, vos diagrammes seront identiques sur le fond, avec les couleurs par défaut. Voici un extrait suffisant pour reprendre l'esprit graphique.

SKIN.IUML (EXTRAIT)

```
skinparam backgroundColor transparent
skinparam shadowing false
skinparam roundCorner 10
skinparam defaultFontColor #374151
skinparam ArrowColor #142D4E
skinparam UsecaseBackgroundColor #DBEAFE
skinparam UsecaseBorderColor #2563EB
skinparam ClassBackgroundColor #FFFFFF
skinparam ClassBorderColor #142D4E
skinparam ClassHeaderBackgroundColor #DBEAFE
skinparam SequenceParticipantBackgroundColor #142D4E
skinparam SequenceParticipantFontColor #FFFFFF
skinparam ActivityBackgroundColor #DBEAFE
skinparam ActivityBorderColor #2563EB
skinparam StateBackgroundColor #DBEAFE
skinparam StateBorderColor #2563EB
skinparam ComponentBackgroundColor #DBEAFE
skinparam NodeBackgroundColor #F3F4F6
```

C Bibliographie et ressources

Références et documentations officielles à consulter. Vérifiez les versions en vigueur.

Référence	Utilité
Royce, W. W. (1970). « Managing the Development of Large Software Systems ». Proceedings of IEEE WESCON.	Article fondateur associé au modèle séquentiel par phases.
Bell, T. E., & Thayer, T. A. (1976). « Software Requirements: Are They Really a Problem? ». Proceedings of the 2nd International Conference on Software Engineering.	Travail souvent cité à propos de l'emploi du terme « waterfall ».
Beck, K. et al. (2001). Manifeste pour le développement logiciel agile. agilemanifesto.org	Valeurs et principes agiles.
Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. scrumguides.org	Définition officielle de Scrum.
Sommerville, I. Software Engineering. Pearson (éditions successives).	Manuel de référence sur les processus et l'ingénierie logicielle.
Pressman, R. S. Software Engineering: A Practitioner's Approach. McGraw-Hill (éditions successives).	Manuel de référence : modèles de processus, tests, qualité.
ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering.	Norme sur l'ingénierie des exigences.
IEEE Computer Society. SWEBOK Guide (Guide to the Software Engineering Body of Knowledge).	Cadre de référence des connaissances en génie logiciel.
Object Management Group. Unified Modeling Language (UML) Specification. uml.org	Spécification officielle d'UML.
PlantUML. Documentation officielle. plantuml.com	Syntaxe et exemples des diagrammes.
Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Thèse de doctorat, University of California, Irvine.	Origine du style d'architecture REST.

Référence	Utilité
Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, IETF.	Spécification des JWT.
OWASP Foundation. OWASP Top 10. owasp.org	Risques de sécurité applicative les plus courants.
Chacon, S., & Straub, B. Pro Git. git-scm.com/book	Livre libre sur Git.
Documentations officielles : developer.mozilla.org (MDN), react.dev, nodejs.org, expressjs.com, dev.mysql.com/doc, docs.docker.com	Technologies utilisées dans le fil rouge.

Le projet fil rouge SupportIT, l'entreprise Atlas Services, les personas, les données et les résultats de tests de ce guide sont **fictifs** et pédagogiques.

Mahdi Cherif — M.D.S — Created by Mahdi Cherif
M.D.S — PFE SERIES — ÉDITION 02
Tous droits réservés — M.D.S — Mahdi Cherif

M.D.S

CREATED BY MAHDI CHERIF

Réaliser son projet PFE — la méthode en cascade (Waterfall Model).

Tous droits réservés — M.D.S — Mahdi Cherif